



Ben-Gurion University of the Negev

Faculty of Engineering Science

School of Electrical and Computer Engineering

Dept. of Electrical and Computer Engineering

Fourth Year Engineering Project

Final Report

Design and Implementation of a Hovering Autonomous Underwater Vehicle (HAUV)

Project number:	p-2026-126
Student:	Matty Rotman - 314834367
Supervisor:	Hugo Guterman
Submitting date:	July 26th, 2026

Table of Contents

Abstract.....	3
Abstract (Hebrew).....	4
List of Abbreviations.....	5
Project Goals - Project Deliverables and Scope Definition	6
Project Deliveries.....	7
Introduction.....	8
Final Technical Specifications	10
Approach Taken to Solve the Problem and Engineering Design	15
Simulation Verification (ROS 2 Gazebo).....	20
Final Acceptance Tests.....	25
Problems and Solutions.....	29
Recommendations and Conclusions	31
References.....	33
Appendices.....	34

Abstract

Autonomous Underwater Vehicles (AUV) are self-contained robots that are capable of executing complex tasks in underwater environments with little to no direct human interactions.

Unlike conventional submarines, AUVs eliminate the need for complex human support systems, enhancing their efficiency and accessibility while providing superior maneuverability compared to submarines of comparable size.

The main goal of the project is to develop an unmanned hovering autonomous underwater vehicle, which, compared to its off-the-shelf counterparts, will be designed for improved accuracy and higher speeds.

The AUV will incorporate a leader-follower architecture, an operator using an off the shelf command and control system will assign the AUV an objective, a location, and the AUV using its processing units will navigate to its designated destination in the shortest possible time while keeping its balance while conducting the required measurements using its onboard sensors, sending them real time to its operator.

Ultimately, a proof of concept (POC) was realized, showcasing an AUV developed as a proof of concept toward open-sea operation while being easily controlled with minimal human intervention.

Keywords: HAUV, Robotics, Command and Control, sensors, Autonomous, Data Processing, Stability Control, User Experience, Maneuverability, Sea Exploration.

תקציר

כלי רכב תת-ימיים (AUV) הם רובוטים עצמיים המסוגלים לבצע משימות מורכבות בסביבות ימיות עם מעט מאוד אינטראקציה ישירה עם בני אדם.

בניגוד לצוללות קונבנציונליות, AUV אינם דורשות מערכות תומכות חיים מורכבות, מה שמיעל אותן ומנגיש אותן תוך מתן יכולת תמרון עדיפה בהשוואה לצוללות בגודל דומה.

מטרת הפרויקט היא פיתוח כלי רכב תת-ימי אוטונומי מרחף בלתי מאויש (HAUV) אשר בהשוואה למקביליו המסחריים ייועד לדיוק גבוה יותר ולמהירויות גבוהות יותר.

ה-AUV בעל ארכיטקטורה מסוג Leader-Follower, בה מפעיל משתמש במערכת שליטה ובקרה (ש"ב) מסחרית יקצה משימה ו-AUV על ידי שימוש במחשבים שלו ינווט ליעד המיועד בזמן הקצר ביותר תוך שמירה על יציבותו וביצוע המדידות הנדרשות באמצעות החיישנים המובנים שלו ולבסוף ישלח את הנתונים בזמן אמת למפעיל.

לבסוף, הושגה הוכחת היתכנות (POC) המדגימה AUV שפותח לקראת פעילות בים הפתוח, תוך שמירה על יכולת שליטה פשוטה ומינימום התערבות אנושית.

מילות מפתח: HAUV, רובוטיקה, שליטה ובקרה, חיישנים, אוטונומיות, עיבוד נתונים, בקרת יציבות, חווית משתמש, יכולות תמרון, חקר ימי.

List of Abbreviations

Abbreviation	Meaning
AUV	Autonomous Underwater Vehicle
BAR100	Blue Robotics depth and pressure sensor
BME280	Bosch temperature, humidity and pressure sensor
BNO055	Bosch 9-axis inertial measurement unit
CAD	Computer-Aided Design
CG	Centre of Gravity
DVL	Doppler Velocity Log
ESC	Electronic Speed Controller
ESP32	Real-time microcontroller (motor and sensor control)
GPS	Global Positioning System
HAUV	Hovering Autonomous Underwater Vehicle
PC	Inter-Integrated Circuit (sensor bus)
IMU	Inertial Measurement Unit
MAVLink	Micro Air Vehicle Link (telemetry and command protocol)
PDR	Preliminary Design Review
POC	Proof of Concept
PWM	Pulse-Width Modulation
QGC	QGroundControl (ground-control station)
ROS 2	Robot Operating System 2
ROV	Remotely Operated Vehicle
T200	Blue Robotics thruster
TF	Transform (ROS coordinate-frame tree)
UART	Universal Asynchronous Receiver/Transmitter
UDP	User Datagram Protocol
URDF	Unified Robot Description Format
USBL	Ultra-Short Baseline (acoustic positioning)

Project Goals - Project Deliverables and Scope Definition

Building on the existing HAUV platform, this continuation project pursued the three goals defined in the preliminary design, restated here in the terms used for the final evaluation.

Goal 1 — Autonomous Mission Execution

Goal: Perform a complete autonomous operation without direct human intervention.

Success criteria: The system successfully executes a complete area scan and marks a specific target in the final experiment while in autonomous-mission mode.

Goal 2 — Operation Stability and Precision

Goal: Maintain high stability and navigation accuracy throughout the mission.

Success criteria: The system holds the key parameters, depth stability, mission-tracking accuracy and object-detection reliability, within the required operational thresholds.

Goal 3 — Real-Time Control and Telemetry

Goal: Ensure continuous monitoring and a live data flow between the HAUV and the operator.

Success criteria: The system successfully measures and transmits real-time data from the onboard sensors to the operator.

Project Deliveries

- Software and control system, a high-level control system running on the UP Board under the ROS 2 framework, together with a real-time control system on an ESP32 microcontroller for motor and sensor management.
- Simulation environment, a complete simulation of the HAUV on the ROS 2 Gazebo simulator, for validating movement, sensor data and control algorithms before physical implementation.
- Physical proof of concept, a functional HAUV integrating sensors, motor control and communications, aimed at navigation and stabilisation.

In Scope

- Development of stability and navigation control algorithms.
- Integration of navigation sensors.
- Validation through simulation and testing.
- Implementation of communication between the HAUV and the operator.

Out of Scope

- Mechanical chassis fabrication, the HAUV's chassis is produced by a separate mechanical team.
- Full waterproofing and open-sea trials, the prototype is not water-resistant, so testing in open-sea conditions was not conducted.

Introduction

Autonomous Underwater Vehicles (AUVs) were first developed in the late 1950s at the University of Washington with the creation of the Self-Propelled Underwater Research Vehicle (SPURV). Weighing 480 kg, SPURV operated at 2.2 m/s for up to 5.5 hours and was primarily used for wave modeling research. Over the following decades, the field matured significantly, transitioning from specialized research platforms to versatile modular systems. Today, AUVs have become a standard and essential tool in academic and industrial research for tasks ranging from seafloor mapping to environmental monitoring [1].

While nowadays AUVs are increasingly affordable and high-performing, commercially available Hovering AUVs (HAUVs) often present limitations for specialized high-speed missions. Popular platforms such as the popular Blue Robotics BlueROV2 are designed as general-purpose systems that prioritize rated depth, operational range, and ease of use over extreme precision or high-tailored maneuvers. This performance gap necessitates the development of our own bespoke solution tailored specifically for improved accuracy and higher speeds. By designing and building a custom HAUV, the project can optimize its thruster configuration and control algorithms to exceed the capabilities of off-the-shelf counterparts [2].

Locating an underwater vehicle with GPS is largely ineffective, because high-frequency electromagnetic waves attenuate rapidly even at shallow depths, and GPS spoofing and jamming add reliability and security concerns. Acoustic methods are used instead: a Doppler Velocity Log (DVL) estimates the vehicle's velocity relative to the seafloor from the Doppler shift of emitted acoustic pulses, and fusing this with inertial measurements yields position over time without surfacing, while acoustic beacons provide absolute positioning by time-of-flight [3]. High-level processing runs on an Intel-based UP Board; ARM platforms such as the Raspberry Pi offer an alternative with lower power and heat and a path to an on-board AI module for computer-vision tasks [4].

Integration of ROS (Robot Operation System) Framework and the Gazebo simulation environment provide framework for development of robotic systems such as the proposed HAUV. A primary advantage of using simulation environment is the facilitation of a seamless transition from software development to physical implementation. As demonstrated in the literature, once robotic models are defined in Gazebo the logic can be easily deployed onto the hardware without significant modifications. Furthermore, simulation enables risk mitigation and performance optimization, by replicating underwater condition within Gazebo, the rigorous testing in software can address potential failures cost-free, eventually validating the HAUV's behavior before it enters the water [5].

To establish a robust command and control link between the HAUV and the operator, it is necessary to bridge the onboard software which manages trajectory and stabilization with the control station that handles operator input and mission planning. The industry standard used to bridge these domains is

the MAVLink protocol - A lightweight, standardized, and vehicle-agnostic system enabling efficient, bidirectional communication of critical telemetry. The protocol is implemented through the QGroundControl (QGC) operator interface. Its open-source and cross-platform architecture is vital, allowing seamless control of the Linux-based HAUV via Windows or even lightweight Android devices. Furthermore, QGC features an extensible mission planning interface that supports bespoke autonomous missions and path-planning algorithms, as required for the HAUV [\[6\]](#).

Building on an earlier bench prototype, a dual-computer, ROS 2-based HAUV run via custom scripts over a wired link with several drivers inoperative, this work adds full autonomy and untethered control: a MAVLink bridge to QGroundControl, an autonomous Guided (go-to-location) mode with depth hold and station-keeping, an acoustic (USBL) link, DVL velocity, and on-board leak, link-loss and attitude failsafes, while retaining the dual-processor split of a Linux UP Board (guidance and telemetry) and an ESP32 (real-time motor and sensor tasks).

Final Technical Specifications

The overall architecture of the HAUV, how power and data are distributed across the vehicle, is summarised in the block diagram below.

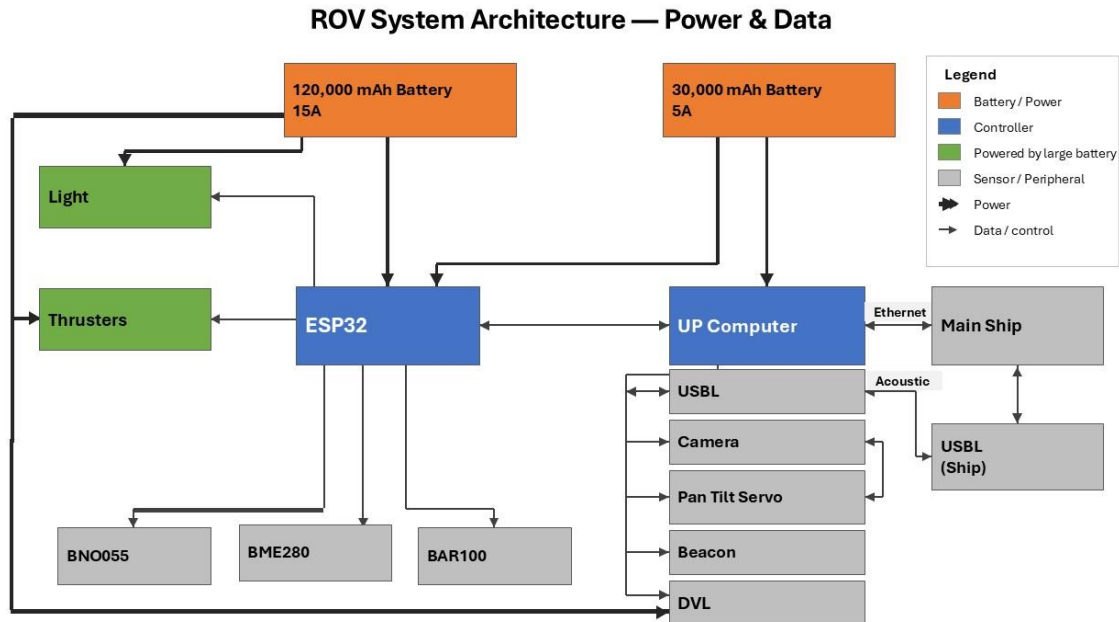


Figure 1 — HAUV system block diagram: power distribution and data/control paths.

The vehicle retains the physical platform and core sensor suite of the earlier stage; the specifications below consolidate that platform with the computing, communication and software additions delivered in this project.

1. Physical Characteristics

- Dimensions: 70 cm x 25 cm x 20 cm
- Frame material: Vslot aluminium profiles
- Manoeuvrability: hovering-class, multiple degrees of freedom (surge, sway, heave, yaw, pitch)

2. Propulsion

- Thrusters: 6 × Blue Robotics T200
- Configuration: two front and two rear thrusters angled inward at 45°, plus one front and one rear vertical thruster
- Motor command: PWM in microseconds, neutral 1500 μ s, full range 1100–1900 μ s; all thrusters initialised to 1500 μ s before operation

3. Sensors

- GPS: u-blox receiver providing a surface position fix for the navigation core
- Depth, temperature & pressure water resistant: Blue Robotics BAR100

- Temperature & humidity: Bosch BME280 electronics monitoring.
- Leak sensor: Blue robotics leak sensor with 4 probes.
- Doppler Velocity Log (DVL): Teledyne Pathfinder for velocity over the seafloor.
- Inertial Measurement Unit (IMU): Bosch BNO055 for orientation data.
- Acoustic position and communication: Advanced Navigation Subsonus.

4. Processing Units

- Main computer, UP Board: Ubuntu 20.04 with ROS 2 Foxy for high-level guidance, sensor fusion, navigation and QGC telemetry
- Real-time microcontroller, ESP32: runs micro-ROS for motor PWM, lights and real-time sensor acquisition.
- Division of labour: the UP Board handles autonomy and communication; the ESP32 guarantees deterministic, low-latency motor and sensor timing.

5. Communication and Networking

- Ethernet (TCP/IP): Primary tethered link. Carries DVL data and general above-/below-water communication through the Blue Robotics Ethernet switch, and transports QGC MAVLink telemetry over UDP.
- Acoustic (USBL): Untethered link over a pair of Subsonus acoustic modems. Carries compact telemetry and operator commands (no video), allowing cable-free operation.
- UART (serial): UP Board ↔ ESP32 over micro-ROS at 115200 baud; a separate FTDI UART is used for ESP32 debug output.
- I²C: ESP32 ↔ all environmental/orientation sensors (BNO055, BAR100, BME280) through a Blue Robotics I2C Bus Splitter board.
- 5 V PWM: Drives the thruster ESCs, camera pan-tilt servo and the lights.

The system uses static IP addresses on the 192.168.168.x/24 subnet, with a common ROS_DOMAIN_ID across all machines. The key devices and endpoints are summarised below.

Device	Address / Port
UP Board (main computer)	192.168.168.101
Doppler Velocity Log (DVL)	192.168.168.102, cmd 1033, binary 1034, string 1037
Operator PC / ground station	192.168.168.100
Subsonus USBL (surface unit)	168.254.1.80
Subsonus USBL (subsea unit)	168.254.1.79

Device	Address / Port
QGC MAVLink, Ethernet	UDP 14550
QGC MAVLink, acoustic bridge	UDP 14551 (127.0.0.1)
ESP32 serial	115200 baud (Silicon Labs CP2102, auto-detected)

Table 1, Network devices and endpoints.

6. Power

- Battery 1: 12 V, 20000mAh lithium pack powering the computing and electronics domain, UP Board, DVL, Ethernet switch and beacon.
- Battery 2: 12 V, 120000mAh lithium pack dedicated to the thrusters and lights, isolating high motor currents from the sensitive electronics.
- DC-DC conversion: A step-down converter provides the regulated 5 V rail for the electronics from the first battery.



Figure 2 — The two 12 V lithium packs and the step-down converter (small green board).

7. Software Architecture and Packages

- Operating system: Ubuntu 20.04 on the UP Board, with ROS 2 Foxy as the robotics middleware coordinating all high-level tasks.
- Microcontroller framework: The ESP32 runs FreeRTOS with micro-ROS (Arduino), exchanging sensor and actuator data with the UP Board over the serial link.

- Operating tool: A single management script, `hauv.sh`, is the one entry point for running and inspecting the stack, start, stop, restart, check (per-sensor OK/LOW/FAIL health at rate), status, topics, echo and view.

The software is organised into ROS 2 packages, each with a well-defined responsibility:

Package	Responsibility
<code>autopilot_pkg</code>	<code>guidance_node</code> (mode switching, PID stabilisation, motor mixing, Guided/GOTO, leak failsafe), <code>dvl_node</code> , <code>subsonus_node</code> , <code>acoustic_bridge_node</code> , <code>health_monitor_node</code>
<code>mavlink_bridge_pkg</code>	Bridges ROS 2 topics to QGroundControl over UDP MAVLink
<code>camera_pkg</code>	USB camera → <code>/camera_image</code>
<code>gps_pkg</code>	u-blox GPS → <code>/gps/fix</code>
<code>my_launch_pkg</code>	Launch-file definitions
<code>esp_sketches/rov_esp_main</code>	ESP32 firmware (micro-ROS), sensor publishing and PWM outputs

Table 2, ROS 2 packages and responsibilities

Data flows between the vehicle's sensors, the guidance core and the ground station through a small set of ROS 2 topics:

Topic	Type	Notes
<code>/esp32/bno055_data</code>	Twist	linear.x/y/z = yaw / pitch / roll (BNO055)
<code>/esp32/bar100_data</code>	Vector3	x = depth (m), y = pressure, z = temperature
<code>/esp32/leak</code>	Float64	0.0 = dry, 1.0 = leak
<code>/dvl/velocity_data</code>	Twist	linear.x/y/z = velocity over ground
<code>/motor_data</code>	Twist	Motors 1–6 as PWM μ s (1100–1900, neutral 1500)

Topic	Type	Notes
/lights_servo_data	Vector3	light 1, light 2, camera-servo angle
/guidance/goto_target	NavSatFix	Guided-mode destination

Table 3, Key ROS 2 topics

Approach Taken to Solve the Problem and Engineering Design

The goal of this stage was to turn the existing HAUV platform into a vehicle that can be operated from a standard ground station, navigate autonomously, and protect itself in a fault. Reaching it meant reuse of the existing hardware, new software subsystems for control and communication, and putting it through rigorous testing.

Structural Overview

The HAUV keeps a dual-computer architecture that cleanly separates deterministic low-level control from higher-level autonomy. The system is composed of three main subsystems:

- Ground station: An operator running QGroundControl on a PC, connected to the vehicle either over the Ethernet tether or over the acoustic (USBL) link.
- Underwater pressure vessel: Houses the core electronics, the UP Board and the ESP32, together with the orientation, depth and environmental sensors.
- Underwater components: The exposed hardware: six T200 thrusters, the lights, the camera, the DVL and the acoustic beacon.

The system splits across two computers, the UP Board (ROS 2) for high-level guidance, navigation and QGC telemetry, and the ESP32 (micro-ROS) for real-time motor, light and sensor control, linked by a guidance_node that fuses the inputs and a bridge node that relays telemetry over Ethernet or the acoustic link.

Key Internal Components

The core electronics are mounted together on the vehicle's electronics tray:

- UP Board, the main computer (identifiable by its large heatsink).
- Indicator lights, red for fault and errors and green for communication issues.
- ESP32, the real-time motor and sensor controller.
- Thruster (one of six Blue Robotics T200 units).
- I²C connector, to the environmental/temperature sensor, via the bus splitter.
- GPS antenna, surface fix used as the origin for navigation.

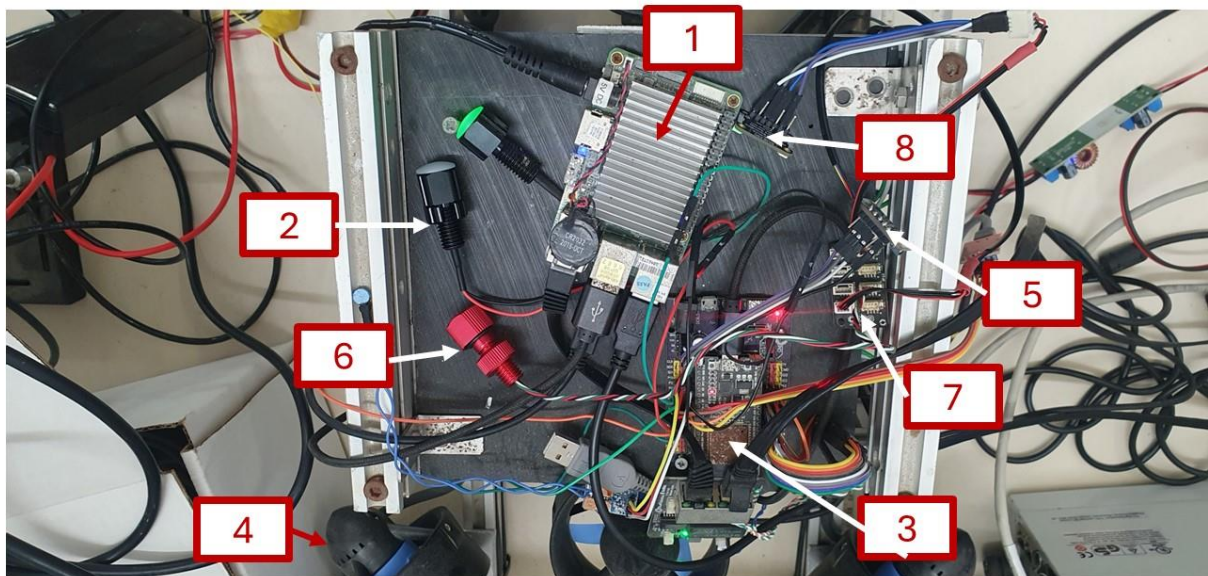


Figure 3 Electronics: (1) UPBoard, (2) indicator, (3) ESP32, (4) thruster, (5) BNO055, (6) BAR100, (7) I2C board, (8) BME.

Doppler Velocity Log (DVL) Integration

Precise underwater navigation depends on an accurate velocity measurement, which cannot be obtained reliably by integrating accelerations over time. A Teledyne Pathfinder Doppler Velocity Log (DVL) was integrated to measure the vehicle's velocity relative to the sea floor directly. The `dvl_node` connects to the DVL over Ethernet (192.168.168.102), parses its output over UDP in its PD6 format and republishes it on `/dvl/velocity_data` for the guidance core and the telemetry bridges. The DVL is configured through its own web interface (Figure 4).

Pathfinder uDVL Network Configuration Update Software Version v1.00

Network Settings

Network Configuration		Link Status
Base IP Address	192.168.168.102	Not Active
Network Mask	255.255.255.0	
Gateway Address	192.168.168.1	
Name Server Address	192.168.168.1	
Command/Control Port	1033	

Output Formats

FORMAT	ENABLE	PROTOCOL	ADDRESS	PORT
PD0	ON	UDP	192.168.168.1	1034
PD4	OFF	UDP	192.168.168.1	1035
PD5	OFF	TCP	192.168.168.1	1036
PD6	ON	UDP	192.168.168.101	1037
PD13	OFF	TCP	192.168.168.1	1038

Submit Reset

Figure 4 — DVL web configuration: static IP 192.168.168.102 and the enabled PD0 / PD6 UDP output formats.

QGroundControl (QGC) Integration

The vehicle was connected to QGroundControl (QGC), a common open-source ground-control station, through a MAVLink bridge node. The bridge is plain Python on ROS 2 with no external MAVLink library, so it is easy to deploy and extend.

- Telemetry: The bridge streams four MAVLink messages to QGC, HEARTBEAT (1 Hz; vehicle alive and armed state), ATTITUDE (12 Hz; roll, pitch and yaw from the BNO055), GLOBAL_POSITION_INT (12 Hz; position and velocity derived from the DVL) and SYS_STATUS (12 Hz; battery, sensors and health).
- Commands: It translates QGC MANUAL_CONTROL (joystick) messages into motor PWM on /motor_data, so the vehicle can be flown directly from the QGC Fly view.

On the operator side the PC is placed on the vehicle subnet (192.168.168.100) and QGC is pointed at a UDP link to 192.168.168.101:14550.

Guided Mode (Autonomous “Go-to-Location”)

In Guided mode, the operator clicks a destination in QGC, and the vehicle autonomously navigates there and holds its position. Because the processing runs locally on the UP Board, a continuous control link is only required to send the target or abort the mission.

This represents the "Autonomous Mission" mode from the preliminary design. (Similarly, "Manual" remains Manual, and the planned "Assisted" mode is now "Stabilize").

When a target is received, the guidance node calculates the required heading and distance, driving the vehicle through three coordinated behaviors:

- Heading and Transit: The vehicle first turns to align with the target before moving forward. Speed is capped for safety and smoothly decreases as the vehicle gets closer.
- Depth Hold: The vehicle locks in its starting depth and maintains it during transit. If triggered at the surface, the vertical thrusters remain neutral.
- Loiter on Arrival: Upon reaching the destination, the vehicle actively maintains its position and depth rather than drifting, waiting for operator takeover.

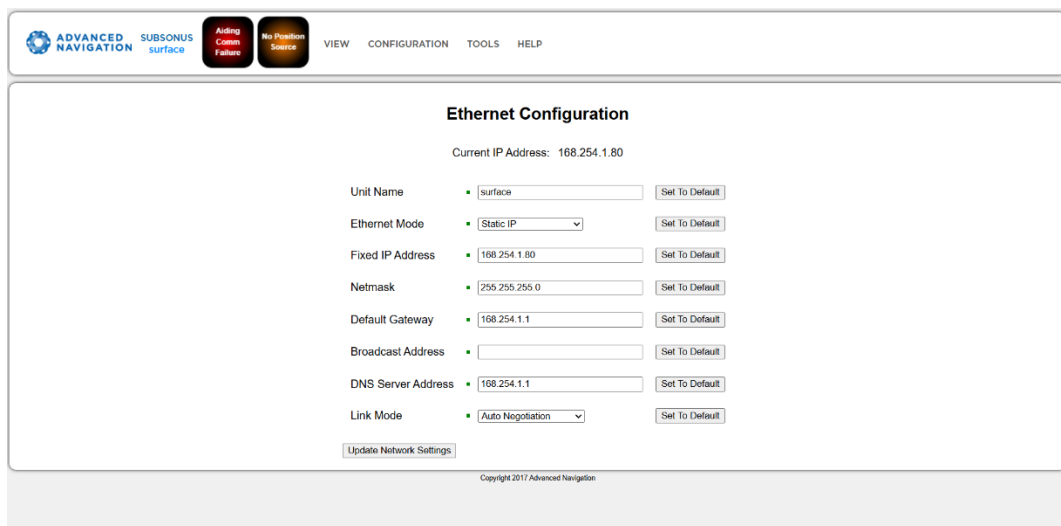
The manoeuvre aborts safely, returning to Manual with neutral thrusters, on any joystick deflection, a switch to Manual mode, loss of the operator link (no input for several seconds), or an implausibly distant target.

Acoustic (USBL) Control Link

To allow cable-free operation, an acoustic control path was built around a pair of Subsonus USBL modems. It mirrors the wired workflow, via a dedicated python script, The bridge carries compact telemetry and the full command set over the acoustic link; video is intentionally omitted, as it is not feasible over the limited acoustic bandwidth.

- Vehicle side: acoustic_bridge_node packages telemetry and unpacks commands over the modem link.
- Operator side: a small bridge program relays between QGC's MAVLink UDP and the acoustic packets, and announces link loss to the operator.

Both modems must be submerged to communicate. The modems are configured through their Ethernet web interface ([Figure 5](#)), where each unit is given a static address on its own subnet.



The screenshot shows the 'Ethernet Configuration' page for a 'surface' unit. The interface includes a top navigation bar with 'ADVANCED NAVIGATION', 'SUBSONUS surface', and status indicators for 'Aiding' (green) and 'No Position Source' (red). The main configuration area lists various network settings, each with a text input field and a 'Set To Default' button. The 'Current IP Address' is displayed as 168.254.1.80. The settings are as follows:

Field	Value	Action
Unit Name	surface	Set To Default
Ethernet Mode	Static IP	Set To Default
Fixed IP Address	168.254.1.80	Set To Default
Netmask	255.255.255.0	Set To Default
Default Gateway	168.254.1.1	Set To Default
Broadcast Address		Set To Default
DNS Server Address	168.254.1.1	Set To Default
Link Mode	Auto Negotiation	Set To Default

An 'Update Network Settings' button is located at the bottom of the configuration area. The footer indicates 'Copyright 2017 Advanced Navigation'.

Figure 5 — Subsonus USBL Ethernet configuration (surface unit).

Safety Systems

A central aim of this stage was for the vehicle to protect itself. Three independent safety mechanisms were implemented, all designed to remain effective even if the link is degraded or lost.

- Leak → auto-surface: On a confirmed leak the guidance node latches an emergency ascent that overrides every other mode, stopping motion and driving the vertical thrusters up to the surface. It runs on the vehicle itself, so it works even if the link drops, and it alerts the operator with a repeated QGC warning.
- Link-loss failsafe: If the operator link goes quiet for more than 5 seconds, the thrusters are commanded to neutral, so a lost connection cannot leave the vehicle driving away on its own.
- Orientation Limits: Commanded thrust is scaled back when pitch or roll become extreme, reducing the risk of the vehicle tumbling or losing controllability in an unstable attitude.

Design Considerations

- Interoperability: Building on MAVLink and QGroundControl, with a dependency-free Python bridge, lets the vehicle be operated with standard, well-supported tooling and keeps the integration easy to extend (e.g. video, missions, parameter tuning).
- Separation of concerns: Real-time motor and sensor tasks run on the ESP32, while autonomy and telemetry run on the UP Board, placing deterministic control and higher-level computation each where it belongs.
- Autonomy on the vehicle: Guided navigation and the leak failsafe execute on-board, so they continue to work when the communication link is degraded or cut.
- Safety-first defaults: Failsafes fail toward the surface and toward neutral thrust, and the vehicle is motion-free at the surface, making bench and shallow-water testing safe.

Simulation Verification (ROS 2 Gazebo)

Vehicle CAD Model

A full 3D CAD model of the HAUV was built in Autodesk Fusion. It uses off-the-shelf watertight enclosures from Blue Robotics for the electronics and batteries, so those parts can simply be bought, while the rest of the vehicle, the frame, trays and mounts, is simple and can be fabricated easily. This model is the proof-of-concept design for the vehicle, and it was the basis for the Gazebo simulation.

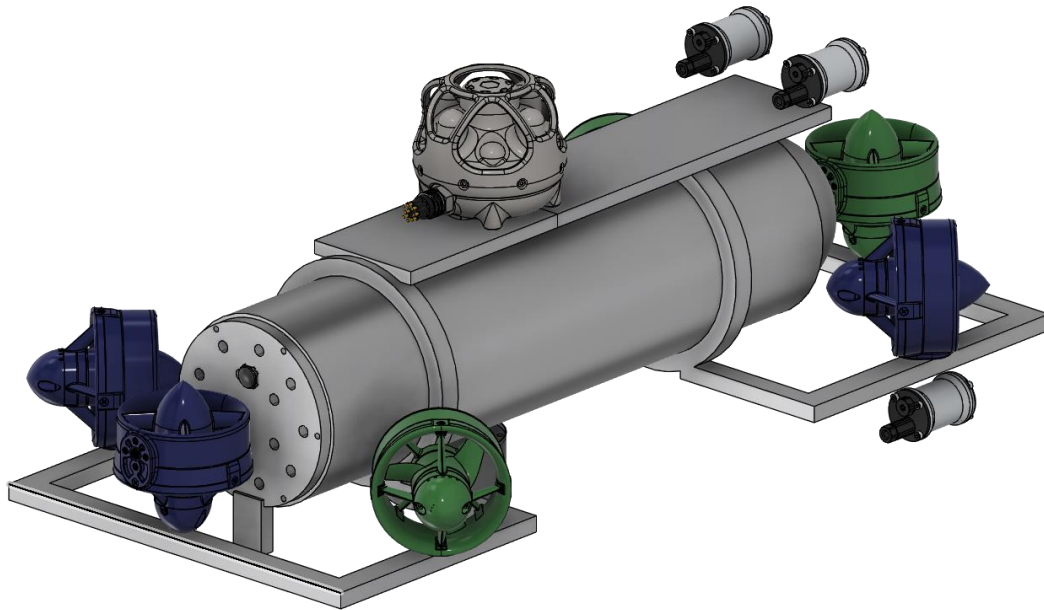


Figure 6 — 3D CAD model of the HAUV (Autodesk Fusion 360).

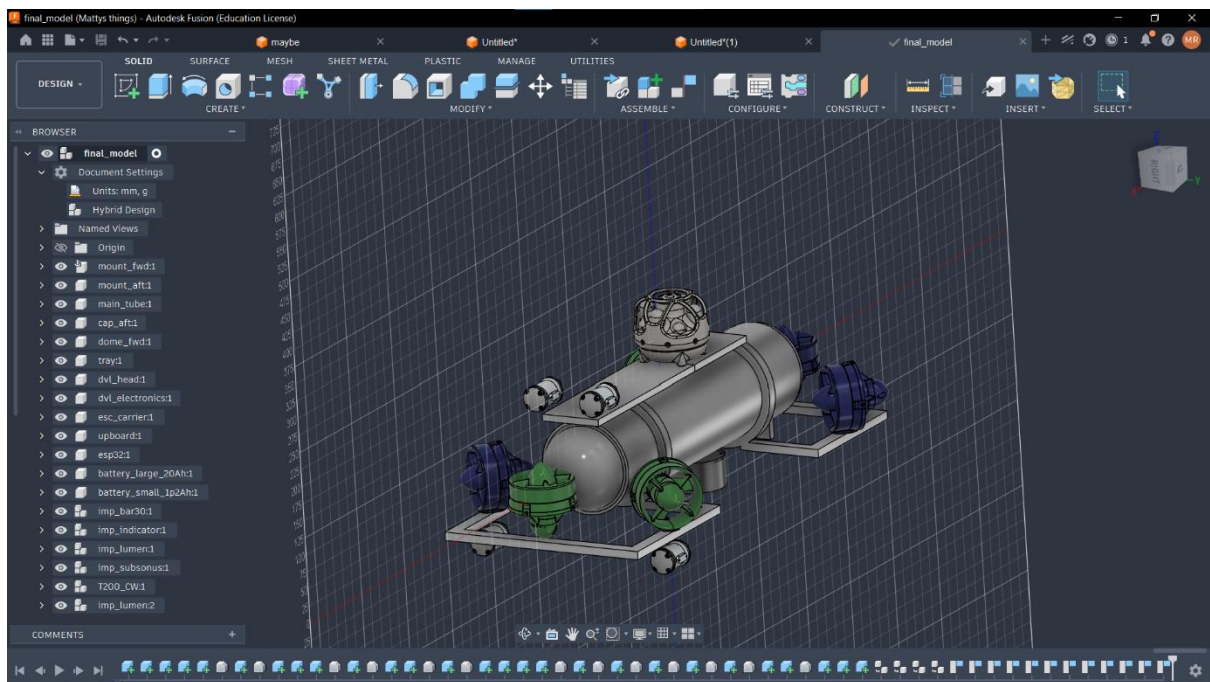


Figure 7 — The CAD assembly in Fusion 360, showing all the vehicle components.

Before any hardware testing, the control software was tested in a Gazebo simulation of the HAUV. The same guidance_node that runs on the real vehicle was flown against the model: bridge nodes fed it simulated sensor data and turned its motor commands into thruster forces, so the code could not tell it was not on the real vehicle. A scripted operator stood in for the operator, and every result below comes from recorded data.

The software was exercised across four scenarios, manual driving, depth hold, autonomous go-to-location and the leak failsafe, and of 24 automated checks, 23 passed. The single failure is deliberate and records a real defect in guidance_node (out-of-range joystick PWM) rather than a problem with the simulation.

Manual Driving

Each stick axis was driven to full deflection in turn. Every command produced motion on its own axis with negligible cross-coupling: forward gave 0.99 m/s of surge, sideways movement reached 1.00 m/s to port, and a yaw command produced 136 °/s of rotation rather than sideways drift, confirming the corrected motor mapping.

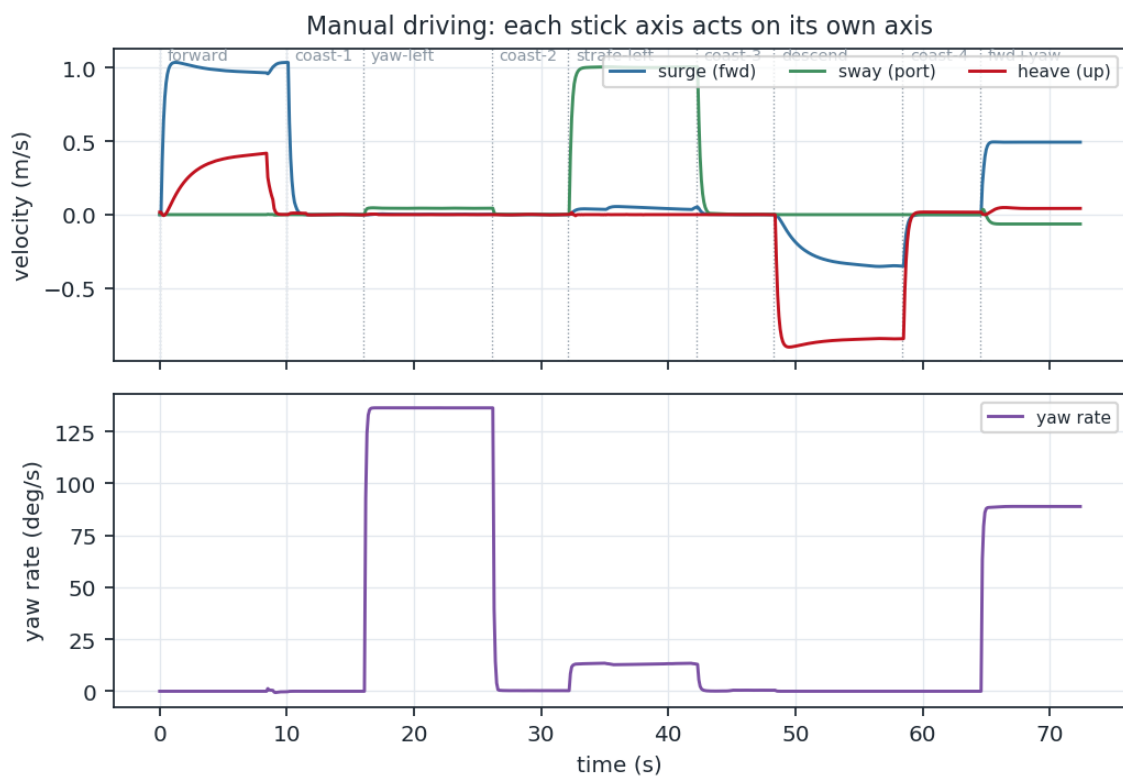


Figure 8 — Body-frame velocities and yaw rate through the manual sequence. Each command acts on its own axis.

Depth Hold

When commanded to hold station at a depth of 6 meters, the vehicle successfully isolated the vertical control loop. Over a 90-second period, it maintained depth within 0.30 m of the target. This exactly matches the controller's no-thrust deadband, representing the tightest tolerance possible for this setup, and successfully countered the 0.90 m of natural buoyant rise that would have occurred unassisted. Additionally, the mean vertical thruster output was 1493 μs (actively pushing downward), verifying that the depth-hold gain sign is correctly configured.

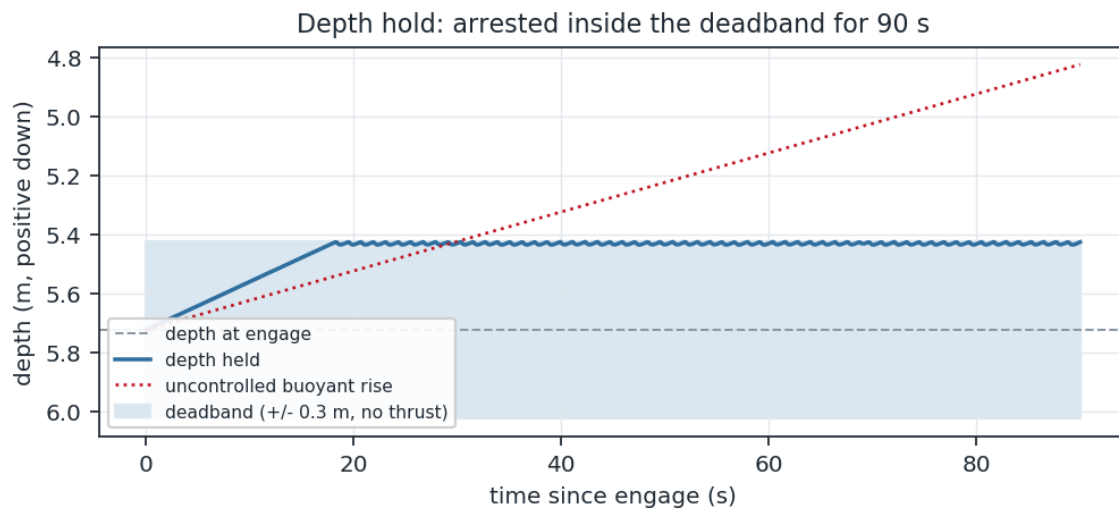


Figure 9 — Depth held inside the deadband over 90 s; the dotted line is the uncontrolled buoyant rise.

Autonomous Go-to-Location

When directed to a target 29.1 m behind its starting position, the vehicle took the shortest turn and smoothly transited to within 1.49 m in 48 seconds without backtracking. The depth varied by just 0.37 m while under way, proving the effectiveness of the depth-hold controller during dynamic maneuvers. The vehicle successfully held station within the 4 m arrival radius for 60 seconds before a manual joystick input cleanly aborted the operation, resetting all motors to a neutral 1500 μ s.

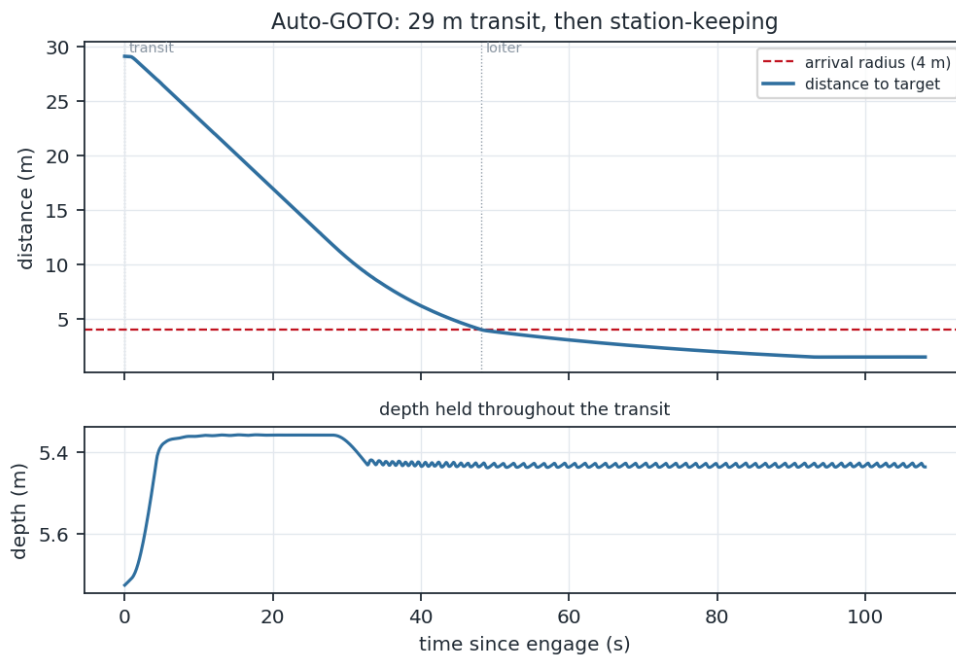


Figure 10 — Distance to target and depth: a 29 m transit in ~48 s, then station-keeping with depth held.

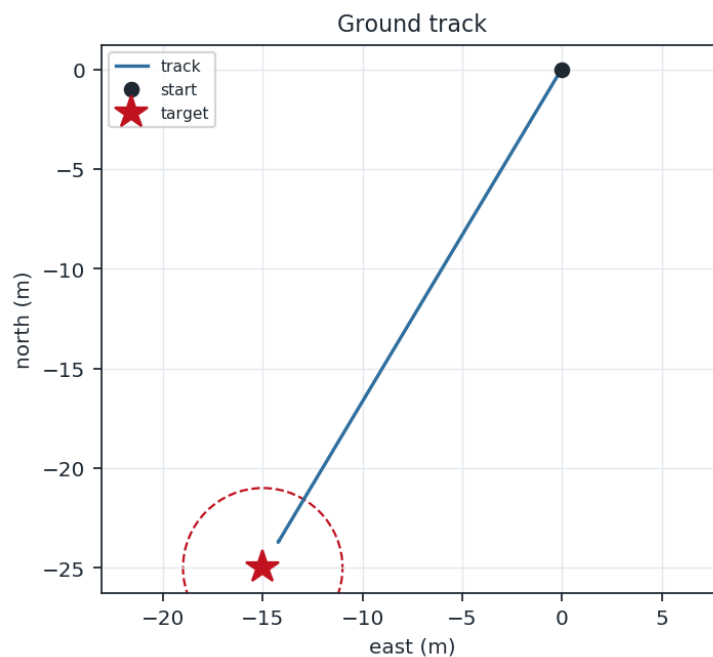


Figure 11 — Ground track in the north-east plane, closing inside the 4 m arrival radius.

Leak Failsafe

A leak was asserted at 7.7 m depth. The failsafe latched after the required two consecutive wet samples and logged “LEAK DETECTED, auto-surfacing”. The horizontal thrusters went to exactly $1500\ \mu\text{s}$ and the verticals to $1820\ \mu\text{s}$ (the commanded 0.8 ascent) until the vehicle reached 0.06 m, whereupon thrust was released and it held at the surface; five seconds of dry readings then auto-cleared it back to manual. The ascent direction proved correct, the vehicle surfaced rather than dived.

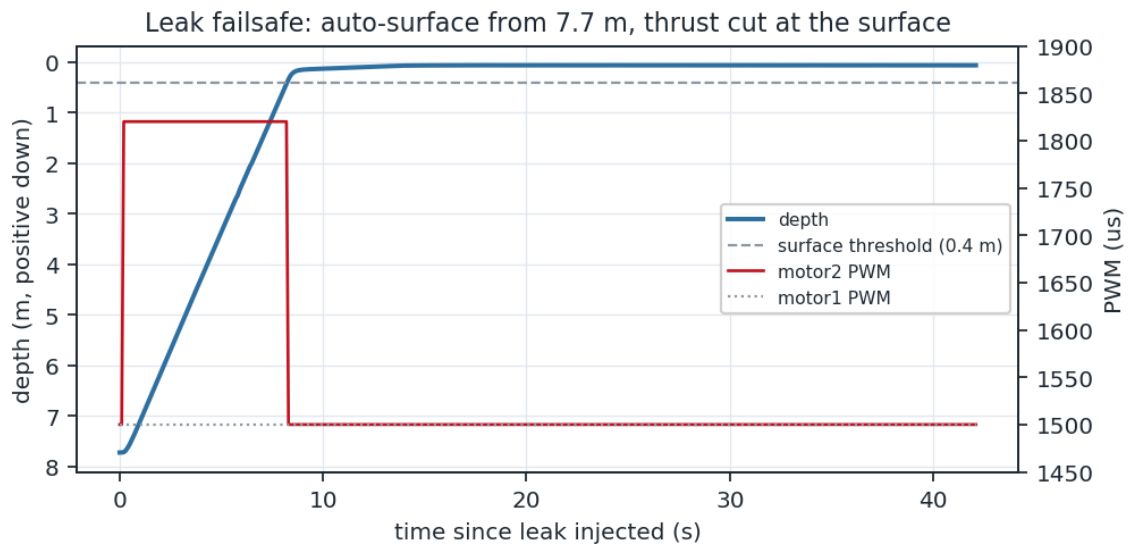


Figure 12 — Leak failsafe: auto-surface from 7.7 m, with thrust cut at the surface.

The Simulated Environment

The figure below shows the HAUV moving at set speed in the simulated water column, a graph can be seen in the right of the angular velocity of the HAUV.

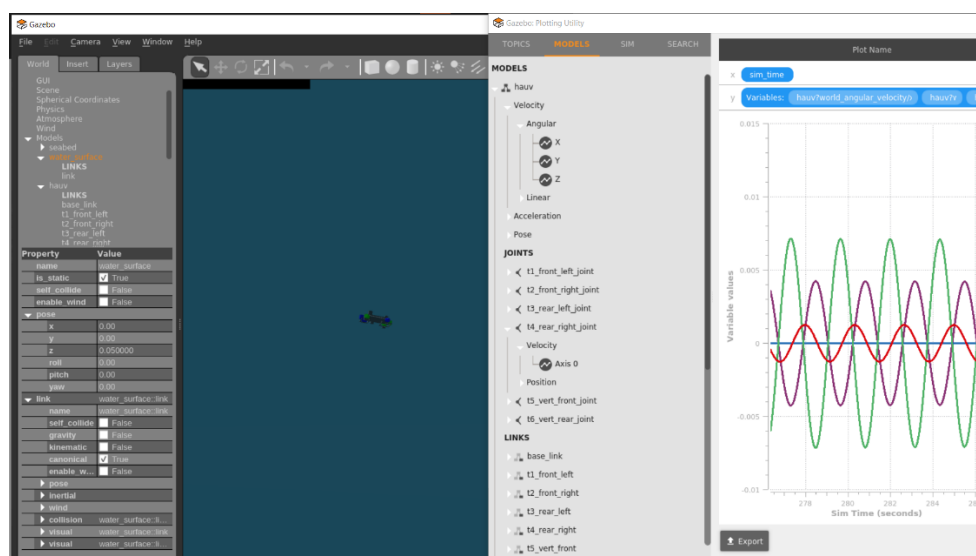


Figure 13 — The HAUV moving in the simulated Gazebo water column.

Findings and Limitations

The Gazebo model is a proof-of-concept designed solely to validate the software and telemetry pipeline, not to simulate real-world hydrodynamics. Performance metrics reflect the simulation's tuning, not the physical HAUV. The guidance_node successfully executed manual driving, depth hold, autonomous GOTO, and leak failsafes end-to-end using the same code intended for the hardware.

Final Acceptance Tests

This section describes the experiments used to validate the integrated system, the telemetry link, the sensors, the autonomous Guided mode, the safety failsafes and the acoustic channel. Testing was carried out in the laboratory and on the bench; full underwater testing remains for a future, water-resistant build. Table 4 summarises every communication link, the level it has been verified to, and what remains to be tested in the water.

Link / component	Interface & protocol	Data (direction)	Verified level	Still to be tested
ESP32 ↔ UP Board	micro-ROS, USB serial 115200 baud	Sensor telemetry up (/esp32/*); motor & light cmds down	Bench-verified: topics stream, commands drive thrusters/lights	Endurance under full load in water
IMU, depth environment sensors → ESP32	I2C	Attitude, depth/pressure, temp/humidity	Bench-verified: IMU ~15 Hz, depth & environment read	
Leak sensor → ESP32	GPIO digital line	Dry/wet flag	Bench-verified: wet-finger test triggers auto-surface	
DVL ↔ UP Board	Ethernet 192.168.168.102 (ports 1033/1034/1037)	Body-frame velocity (/dvl/velocity_data)	Link & parsing confirmed	
Joystick → UP Board	USB Xbox controller → /joy	Pilot input (all axes + buttons)	Working: drives every axis on the bench	Field ergonomics only
UP Board ↔ QGC (Ethernet)	MAVLink over UDP 14550	Heartbeat, Position, Status & Control	Bench-verified: auto-connects, 4 telemetry streams at rate, joystick control	Range & latency in the field

Link / component	Interface & protocol	Data (direction)	Verified level	Still to be tested
Camera → UP Board → QGC	USB camera → H.264 video	Live video stream	Bench-verified: live video in QGC over Ethernet	
UP Board ↔ QGC (Acoustic)	Subsonus modem + acoustic bridge (telemetry, no video)	Telemetry + commands + leak byte	Bridge verified: packets encode/decode, leak alert forwarded	End-to-end acoustic link in water
Guided mode	Route QGC MAVLink data to the appropriate topic.	Target lat/lon	Bench-verified	In-water navigation
GPS → UP Board	NMEA → /gps/fix (surface)	GPS coordinations	Bench-verified: fix feeds the guidance core	

Table 4, System communication summary.

The vehicle's live state was also visualised in RViz: its URDF model, driven by the real-time TF tree from the on-board sensors, gives a 3D view of the HAUV's orientation and thruster layout during operation ([Figure 14](#)).

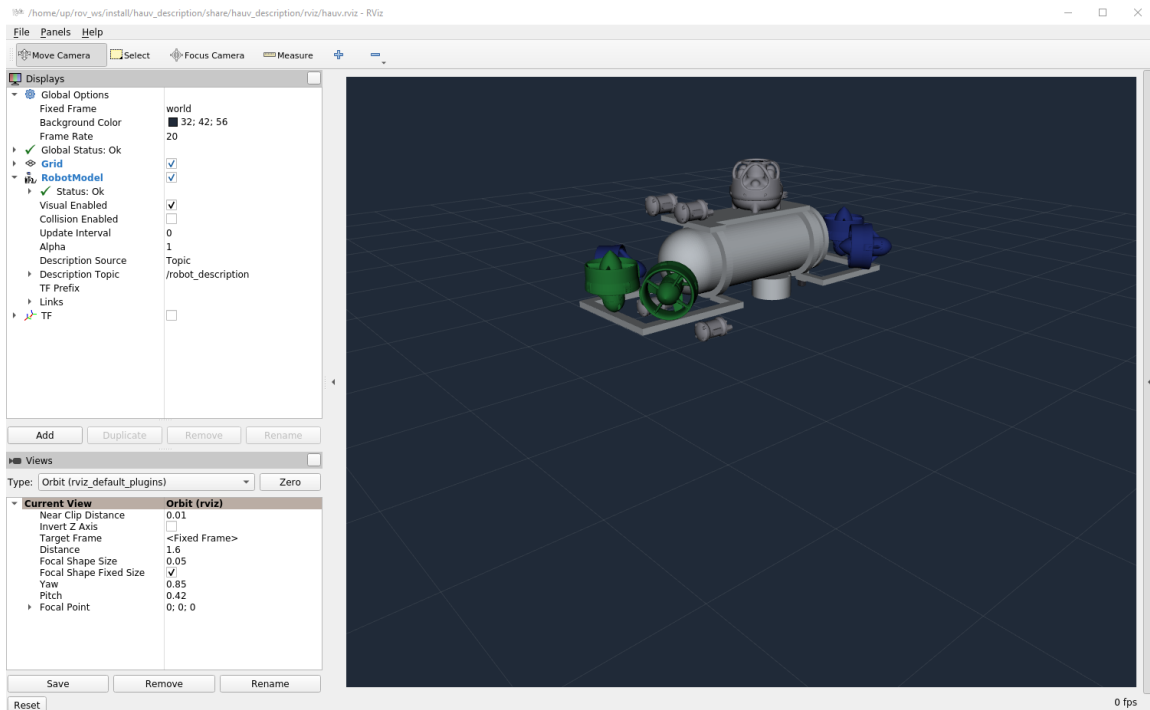


Figure 14 — Live RViz visualisation of the HAUV, driven by the vehicle's URDF model and real-time TF tree.

1. QGroundControl Telemetry and Control

- Telemetry: With QGC connected over Ethernet, the MAVLink Inspector confirmed all four message types arriving at their design rates, HEARTBEAT at ~1 Hz and ATTITUDE, GLOBAL_POSITION_INT and SYS_STATUS at ~12 Hz. The QGC attitude indicator tracked the BNO055 as the vehicle was tilted by hand.
- Control: Joystick input in the QGC Fly view was translated by the bridge into motor commands on /motor_data, and the thrusters responded in real time, confirming the command path QGC → bridge → guidance → ESP32 → motors.

2. System Health Check

The hauv.sh check command samples every sensor topic and reports OK / LOW / FAIL together with the measured rate by sampling the data in a 4 second window, giving a one-command view of system integrity. A representative result showed the IMU, environmental and leak topics all reporting at ~12 Hz, while the DVL and USBL report at at ~20 Hz.

3. Doppler Velocity Log

The DVL was configured through its web interface (network settings and enabled output formats) and its velocity output verified on the /dvl/velocity_data topic, confirming correct parsing and republishing by dvl_node for use by the guidance core and the telemetry bridges.

4. Autonomous Guided Mode (bench-safe)

- Navigation: On a QGC “Go-to-location” command, the vehicle entered Guided mode, computed the distance and heading from its GPS coordinates to the target, and aligned itself with the correct direction before driving forward, confirming the full command chain into calculate_goto_motor_speeds.
- Depth hold: Because the depth-hold system keeps the vertical thrusters neutral when the vehicle is at its engaged (surface) depth, the manoeuvre is motion-free vertically on the bench, allowing safe out-of-water testing.
- Aborts: Every abort path was verified to return the vehicle to Manual with motors neutral, any joystick deflection, a switch to Manual mode, a comms timeout, or a target beyond the maximum range.

5. Leak Auto-Surface Failsafe

The failsafe was tested by simulating a leak at the surface. The leak triggered the emergency auto-surface mode, and the system automatically recovered once the sensor dried. Since the vehicle was already at the surface, the vertical thrusters remained neutral, keeping the vehicle completely still during the test. The expected emergency alert was also successfully triggered in QGC.

6. Acoustic (USBL) Link

Over the acoustic bridge, QGC operated the vehicle using the compact telemetry packet, and loss of the acoustic link was displayed to the operator. As the navigation runs on the UP Board, the acoustic link is only needed to send a target, abort and send some sensor data, not throughout the run, which keeps the untethered mode robust to intermittent acoustic conditions.

7. Validation Status and Remaining In-Water Tests

All acceptance tests to date were performed on the bench with the thrusters clear of obstruction. The system is therefore bench-verified end to end; full validation now requires an in-water run. The items below are the remaining tests and their pass criteria. The two navigation sign conventions are each a single-line change, deliberately left to confirm on the first dive rather than guessed on the bench.

- Yaw direction (Guided mode) – vehicle turns toward the target bearing.
- Depth-hold direction (Guided mode) – vertical thrusters hold the captured depth
- Leak auto-surface direction – on an injected leak the vehicle ascends to the surface.
- Safety Envelope Validation – Testing that commanded thrust is automatically scaled back during extreme pitch or roll events on fully in the water.
- Acoustic USBL testing – Testing was delayed because the necessary waterproof cables and proper PoE components were unavailable, and the system requires a larger body of water for accurate acoustic testing.
- Full wet test of the three control modes – Manual, Stabilize and Guided each drive the vehicle correctly in the water.

Problems and Solutions

Several substantial engineering problems had to be solved during this stage, spanning software, sensors, communication and hardware reliability.

1. Broken sensor logic and cluttered ROS communication

- Problem: The inherited codebase had broken sensor logic and a cluttered set of ROS topics, which made the vehicle's state hard to trust and to debug.
- Solution: After building working proficiency in ROS 2, the codebase was refactored: the broken sensor logic was restored, unused topics were pruned, and a dedicated temperature topic was added. This produced a clean, reliable data flow that the new autonomy and telemetry features could build on.

2. Running all sensors at once on a single I²C bus

- Problem: The sensors could not all be operated simultaneously, the initial wiring was limited to three devices, which was insufficient for the full sensor suite.
- Solution: A dedicated I²C breakout board was added, allowing the BNO055, BAR100 and BME280 to share the bus and operate together. A BMP280/BME280 mismatch was corrected in the drivers, and the environmental sensor was later swapped for an Adafruit board after its fixed I²C address was found to clash with the BNO055.

3. Unreliable debugging and power during development

- Problem: An interfering LCD screen and an unstable battery supply made bench development unreliable.
- Solution: The LCD was replaced with a stable UART debug channel, and the development setup was powered from a wall charger instead of the battery, giving a stable and repeatable bench environment.

4. MAVLink Bridge

- Problem: Building a MAVLink bridge that QGroundControl will accept is non-trivial, since QGroundControl is designed for aerial drones rather than underwater vehicles.
- Solution: The MAVLink bridge was written to present the vehicle as an ArduSub device, which unlocks QGroundControl's built-in underwater (submarine) capabilities.

5. Incompatible Python libraries

- Problem: The Subsonus and DVL Python libraries were incompatible with the HAUV's Python version.

- **Solution:** Bespoke scripts were written to communicate with the devices directly, reusing the relevant code from the vendor libraries. This removed the Python-version dependency and left the codebase cleaner and easier to maintain.

6. Persistent ESP32 lock-up

- **Problem:** The ESP32 would intermittently lock up, its indicator LED going solid while all sensor topics died, the moment the software stack started, taking every sensor down with it.
- **Solution:** Two root causes were found and fixed. A failing BAR100 depth sensor was freezing the firmware on its first read (only after the micro-ROS agent connected); and the start-up script's reset pulse never actually reset this board, so it was swapped for a software esptool hard reset, letting the stack restart cleanly without a manual button press.

7. Work halted due to the Iran War

- **Problem:** From March to mid-July the war with Iran closed the campus and called me up for reserve duty, so lab work stopped for several months.
- **Solution:** While the lab was closed I worked on the parts that did not need it, the Gazebo simulation and the CAD model and returned to on-site work once the campus reopened.

Recommendations and Conclusions

Summary of Work

This stage advanced the HAUV from a bench prototype into an operable, autonomous and self-protecting vehicle. It was integrated with QGroundControl through a custom MAVLink bridge, an autonomous Guided (go-to-location) mode was added, an untethered acoustic (USBL) control channel was implemented, a Doppler Velocity Log was integrated for velocity measurement, and on-board leak, link-loss and attitude failsafes were built. These capabilities rest on substantial systems-integration and reliability work: refactoring the codebase, soldering cables, and diagnosing and resolving a persistent ESP32 lock-up. As in the earlier stage, the vehicle has not yet been deployed in a water-resistant enclosure, so live underwater testing remains incomplete.

Comparison of Results with Project Goals

The three project goals were largely met in the laboratory, on the bench, and in simulation. QGC connects automatically and displays all telemetry streams at their target rates, with joystick input driving the thrusters (Goal 3). Guided mode computes the bearing and distance to a selected target and drives toward it, aborting safely on operator input or link loss, demonstrating autonomous point-to-point navigation (Goal 1). Goal 1's success criterion, however, specified a complete area scan that marks a specific target; that area-scan capability was not demonstrated, and autonomy was validated only as point-to-point GOTO on the bench and in simulation. Depth hold and station-keeping held the vehicle within operational thresholds, providing the stability and precision evidence for Goal 2, while mission-tracking accuracy and object-detection reliability remain to be confirmed in water. The acoustic bridge carries the same telemetry and commands without a cable (Goal 3). The leak auto-surface, link-loss failsafe and single-command operation were also demonstrated on the bench. What remains unverified is the full underwater performance that still to be validated.

Advantages and Disadvantages

Advantages:

- Operable with standard, off-the-shelf tools (QGroundControl over MAVLink).
- Two control links, a wired tether with full telemetry and video, and an untethered acoustic link.
- On-board autonomy and failsafes that keep working when the operator link is degraded or cut.
- Modular, refactored software with a single-command operating and diagnostics tool.

Disadvantages:

- Not yet water-resistant, so real-world underwater applicability is unverified.
- Surface station-keeping is as tight as the GPS fix; underwater USBL positioning would be tighter.
- The acoustic link carries only compact telemetry and no video.

Recommendations for Future Work

- Water-resistant enclosure and live underwater trials to validate navigation, depth hold and the failsafes in the water, including the two sign-convention checks.
- Mission planning: extend the MAVLink bridge with waypoint missions and parameter tuning through QGC.
- Video or richer telemetry over the acoustic path through improved compression.

Conclusions

This project turned the HAUV into a vehicle that can be flown from a standard ground station over either a wired or a wireless link, navigate autonomously to a target, and protect itself in a fault.

Combined with a refactored and more reliable software base, the platform is now a solid foundation for the remaining step: a water-resistant build and live underwater validation.

References

1. Ewart, T. E., "Observations from Straight Line Isobaric Runs of SPURV," Joint Oceanography Assembly, Edinburgh (UK), 13 Sep. 1976.
2. Blue Robotics, "BlueROV2 – Underwater ROV." [Online]. Available: <https://bluerobotics.com/store/rov/bluerov2/> (accessed Nov. 15, 2024).
3. J. J. Leonard and A. Bahr, "Autonomous Underwater Vehicle Navigation," in Springer Handbook of Ocean Engineering, 2016, pp. 341–358.
4. R. V. Aroca and L. M. G. Gonçalves, "Towards green data centers: A comparison of x86 and ARM architectures power efficiency," Journal of Parallel and Distributed Computing, vol. 72, no. 12, pp. 1770–1780, 2012.
5. G. Andreescu, C.-D. Căleanu and S. Nanu, "Simulation environment for mobile robots testing using ROS and Gazebo," in Proc. 20th Int. Conf. System Theory, Control and Computing (ICSTCC), Sinaia, Romania, 2016, pp. 329–334.
6. C. Ramirez-Atencia and D. Camacho, "Extending QGroundControl for Automated Mission Planning of UAVs," Sensors, vol. 18, no. 7, p. 2339, 2018.

Appendices

Appendix A — Operating the vehicle (hauv.sh)

The entire software stack is run and inspected through a single management script on the UP Board.

Command	Action
<code>./hauv.sh start [--ethernet --acoustic]</code>	Start the full stack (default: Ethernet / tethered)
<code>./hauv.sh stop</code>	Stop everything
<code>./hauv.sh restart [--ethernet --acoustic]</code>	Stop, then start
<code>./hauv.sh check</code>	Sample every sensor topic and report OK / LOW / FAIL with rate
<code>./hauv.sh status</code>	List the running node sessions
<code>./hauv.sh topics</code>	List live topics with their short names
<code>./hauv.sh echo <NAME>/topic></code>	Stream one topic's values (e.g. echo DEPTH)
<code>./hauv.sh view <node></code>	Attach to a node's log (Ctrl-A D to detach)

Table A1, hauv.sh operating commands

Appendix B — QGC MAVLink telemetry messages

Message	Rate	Source	Fields
HEARTBEAT	1 Hz	Bridge	Vehicle type, armed state, mode
ATTITUDE	12 Hz	BNO055	Roll, pitch, yaw
GLOBAL_POSITION_INT	12 Hz	DVL	Position, velocity, heading
SYS_STATUS	12 Hz	Sensors	Battery, sensor health, error counts
MANUAL_CONTROL (in)	event	QGC	Joystick axes → motor PWM on /motor_data

Table B1, MAVLink messages exchanged with QGroundControl

Appendix C — Connecting QGroundControl

Over Ethernet (normal): set the operator PC to 192.168.168.100 (netmask 255.255.255.0), verify with ping 192.168.168.101, then add a UDP comm link to 192.168.168.101:14550. Turn QGC's UDP AutoConnect off so it does not grab the wrong endpoint.

Over acoustic (untethered): with both Subsonus modems in the water, start the vehicle with `./hauv.sh start --acoustic`, run one instance of the PC-side acoustic bridge, and point QGC at a UDP link to 127.0.0.1:14551. Telemetry is compact and there is no video on the acoustic link.

Appendix D — Project Timeline

The project was carried out by a single student across six phases, from initial design through validation, final testing and a digital-twin simulation. It was also halted for several months by the outbreak of the war with Iran. The timeline is summarised below.

Phase 1 — Initialization and Design (October 2025)

- Define the system requirements.
- Review the code and components available in the lab.
- Learn the ROS framework.
- Milestone: submission of the PDR.

Phase 2 — Simulation and Physical Setup (November–December 2025)

- Set up the software environment (Linux and ROS 2).
- Learn QGroundControl.
- Implement basic control in the simulation.

Phase 3 — Core Development (December 2025–February 2026)

- Develop the drivers and integrate the sensors.
- Develop autonomous navigation with sensor fusion.
- Integrate with QGroundControl in simulation.
- Milestone: readiness of the simulation.

Operation Rising Lion — Development Halted (March–July 2026)

- Reserve-duty call-up and closure of the campus paused the project for several months.

Phase 4 — Hardware Integration and Dry Testing (June 2026)

- Assemble the electronic stack (UP Board, ESP32 and boards).
- ESP32 firmware, indicator LEDs and brown-out handling.
- Dry-test the HAUV with QGroundControl.
- Milestone: readiness of the hardware.

Phase 5 — Validation and Final Testing (June–July 2026)

- QGroundControl integration (telemetry and video).
- Guided mode / Auto-GOTO (depth hold and loiter).
- Acoustic USBL link (Subsonus).
- Leak auto-surface failsafe.
- Write the final report and operator's manual.

Phase 6 — Simulation and Digital Twin (June–July 2026)

- Pressure-enclosure CAD (Fusion 360).
- RViz visualisation model (URDF).
- Gazebo simulation, flying guidance_node in the sim.
- Milestone: digital twin complete.

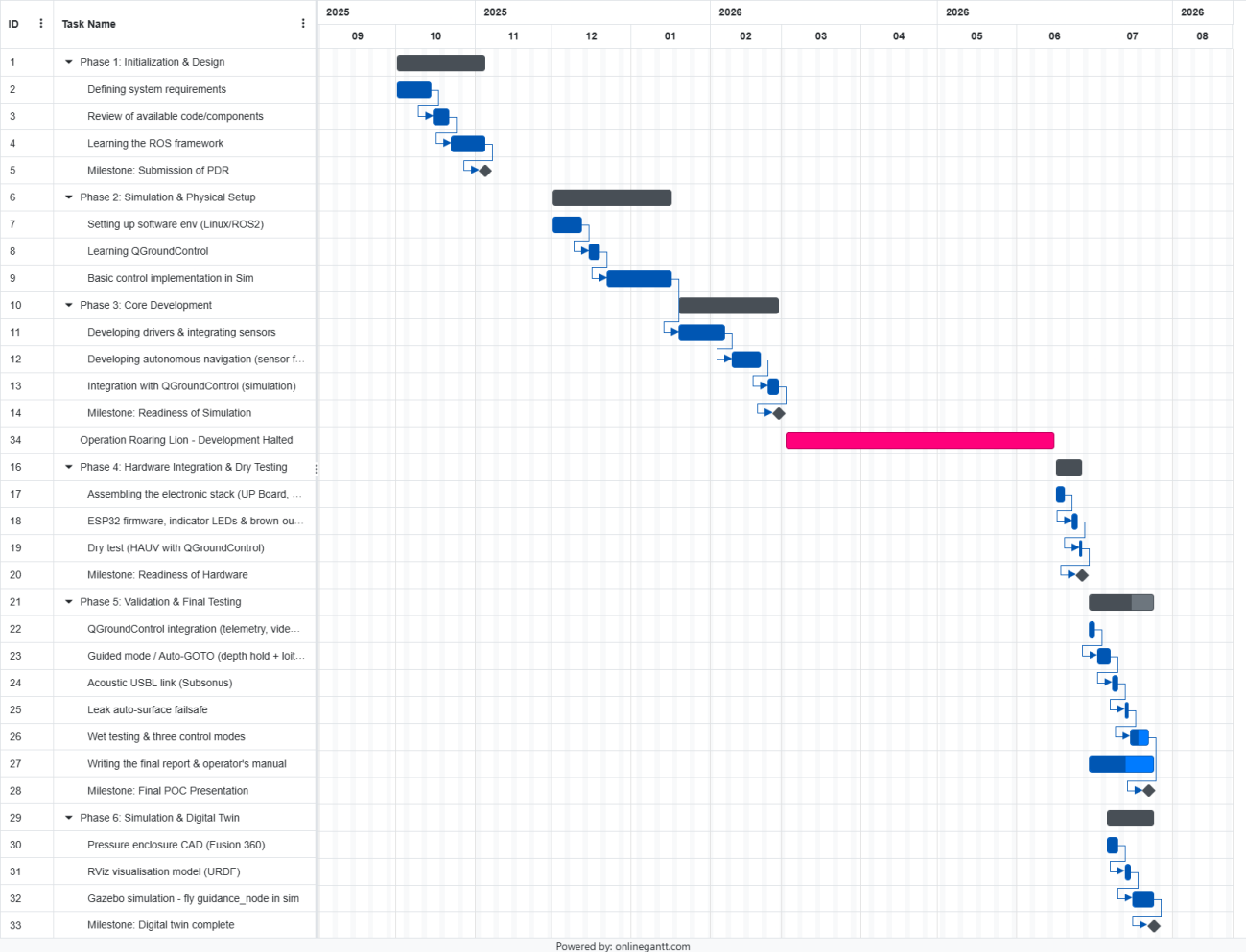


Figure 15 — HAUV Project Development Gantt Chart.

Appendix E — Budget Estimation

The budget estimate covers the critical hardware components required for the HAUV proof of concept. All software used is open-source, so no software expenses are incurred, and a 10% contingency was allocated for unforeseen costs. The total estimated budget is USD 55,625.

Item	Qty	Unit Price	Subtotal	Notes
Subsonus USBL Transponder	2	\$20,000	\$40,000	Advanced Navigation
Doppler Velocity Log (DVL)	1	\$8,000	\$8,000	Teledyne Pathfinder OEM
Thrusters	6	\$200	\$1,200	Blue Robotics T200
Electronic Speed Controllers	6	\$38	\$228	Blue Robotics Basic ESC
Lights	2	\$160	\$320	Blue Robotics Lumen Subsea Light
Ethernet Switch	1	\$175	\$175	Blue Robotics
UP Board	1	\$100	\$100	Main processing unit
Camera	1	\$100	\$100	Blue Robotics Low-Light Camera
Beacon	1	\$80	\$80	Blue Robotics
Pan-Tilt Servo	1	\$64	\$64	Blue Robotics Camera Tilt Mount
Frames	1	\$50	\$50	Aluminium profiles
Leak Sensor	1	\$35	\$35	Blue Robotics SOS Leak Sensor
Battery 2 (120,000 mAh)	1	\$31	\$31	For motors and lights
ESP32 WROOM & 38-Pin Expansion Board	1	\$30	\$30	Real-time microcontroller with additional I/O
Battery 1 (20,000 mAh)	1	\$25	\$25	For UP Board, DVL and other components
Indicator	2	\$15	\$30	Blue Robotics
Miscellaneous	—	—	\$100	Cables, connectors, etc.
Contingency (10%)	—	—	\$5,057	Covers unexpected expenses
Total Budget			\$55,625	

Appendix F — Software Documentation and Source-Code Layout

The complete source code is publicly available on GitHub at <https://github.com/mattyrot/HAUV-Final-Engineering-Project-Mine>. This appendix maps each software component to its location; the package responsibilities and key ROS 2 topics are given in Section 7 (Table 2 and Table 3).

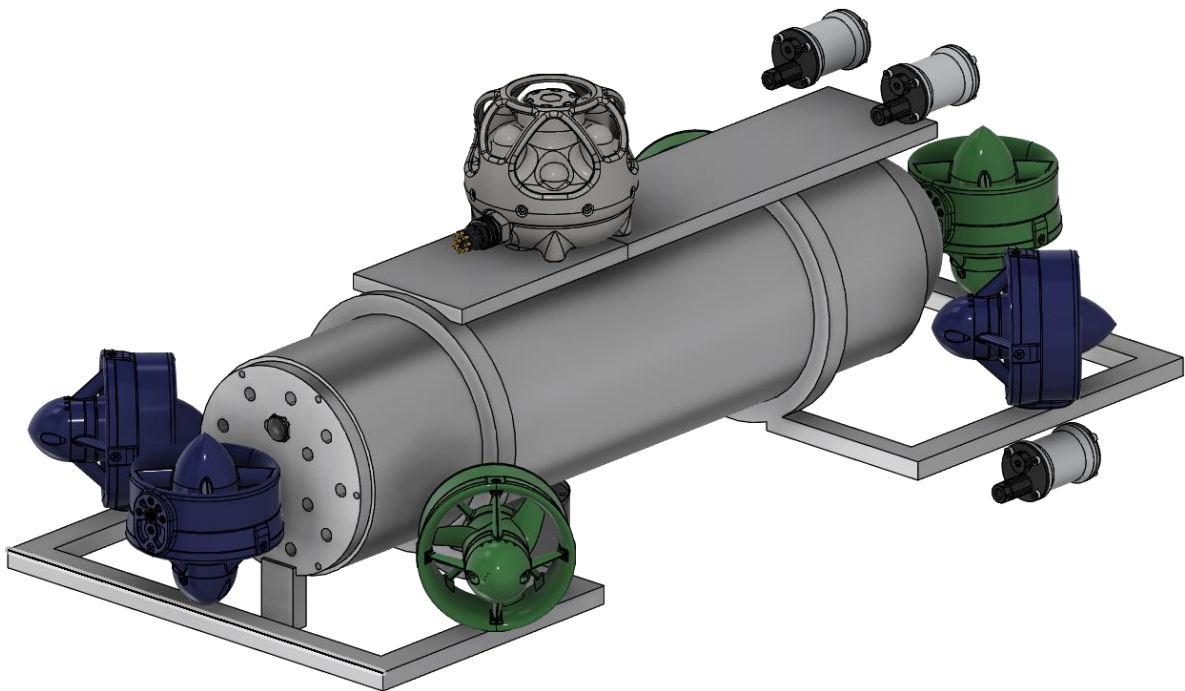
Component / path	Runs on	Responsibility
src/autopilot_pkg	UP Board	Guidance, PID stabilisation, Guided/GOTO navigation, leak auto-surface, DVL driver
src/mavlink_bridge_pkg	UP Board	ROS 2 <-> QGC over MAVLink (Ethernet) and the acoustic USBL link
src/camera_pkg	UP Board	USB camera -> /camera_image video
src/gps_pkg	UP Board	GPS fix for surface navigation
src/my_launch_pkg	UP Board	ROS 2 launch files
src/hauv_description	Workstation	RViz URDF model driven from live attitude + depth
src/hauv_gazebo	Workstation (WSL)	Gazebo digital twin that flies guidance_node unchanged
src/esp_sketches/rov_esp_main	ESP32	Real-time PWM, I2C sensor readout, micro-ROS publisher/subscriber
/tools	UP Board / PC	acoustic_qgc_bridge (PC), DVL plotting, deploy & flash helpers
/docs	-	Final report, operator's manual, project Gantt
/Enclosure	Fusion/Solidworks	pressure-enclosure CAD (STEP) + renders
/pics	-	figures used in the documents
hauv.sh	UP Board	Management tool (start/stop/check/echo/view)

Table F1, Repository layout and component locations.

The workspace is built with colcon build from the repository root. The ESP32 firmware is compiled and flashed with arduino-cli. Python changes are deployed to the UP Board's install tree over SCP. Day-to-day operation is through ./hauv.sh (see Appendix A).

HAUV

Hybrid Autonomous Underwater Vehicle



Operator's Manual

How to set up, drive and look after the HAUV

Draft v1.0
19 July 2026

About the HAUV

What the HAUV is

The HAUV is a small underwater robot. You drive it from a laptop on the shore or on a boat. It can also drive itself to a point on the map.

The two computers inside

The HAUV has two computers. They do different jobs. It helps to know which one does what when something goes wrong.

Computer	What it does
UP Board	The “brain”. It thinks. It talks to your laptop, reads the GPS and the DVL, decides where to go, and runs the safety rules.
ESP32	The “muscles”. It turns the motors, switches the lights, and reads the depth sensor, the compass and the leak sensor.

The two computers talk to each other over a small cable inside the vehicle. If that link breaks, the green indicator light (Figure B, #2) goes off (see Chapter 10).

The main parts

Figure A shows the vehicle with the main parts numbered.

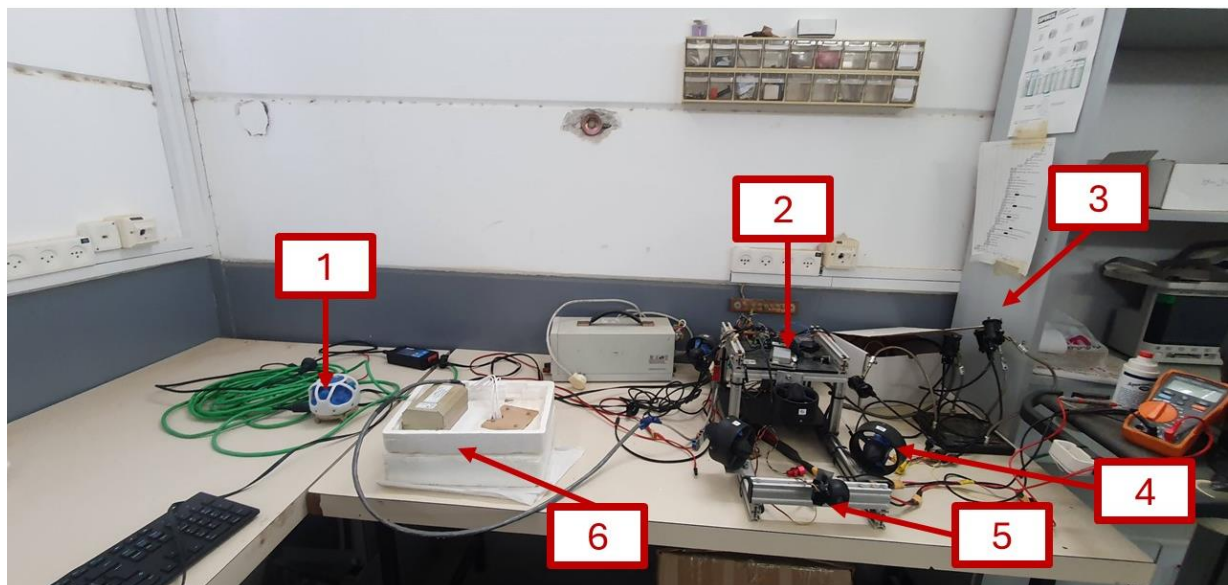


Figure A — The HAUV and its main parts. The numbers match the table below.

#	Part	What it is for
1	Subsonus (USBL modem)	Talks to the surface with sound, so the vehicle can be controlled with no cable.
2	Electronics tray	Holds the two computers and the boards. See Section 1.4.

#	Part	What it is for
3	Lights	Two sets of lamps. You switch them from the controller.
4	Thrusters	The propellers that move the vehicle.
5	Camera	Sends the live picture to your screen.
6	DVL	Measures speed over the sea floor.

Inside the vehicle — the electronics

Figure B shows the electronics tray with the main boards numbered. You do not need to touch these to drive the vehicle. It helps to know where they are when something stops working.

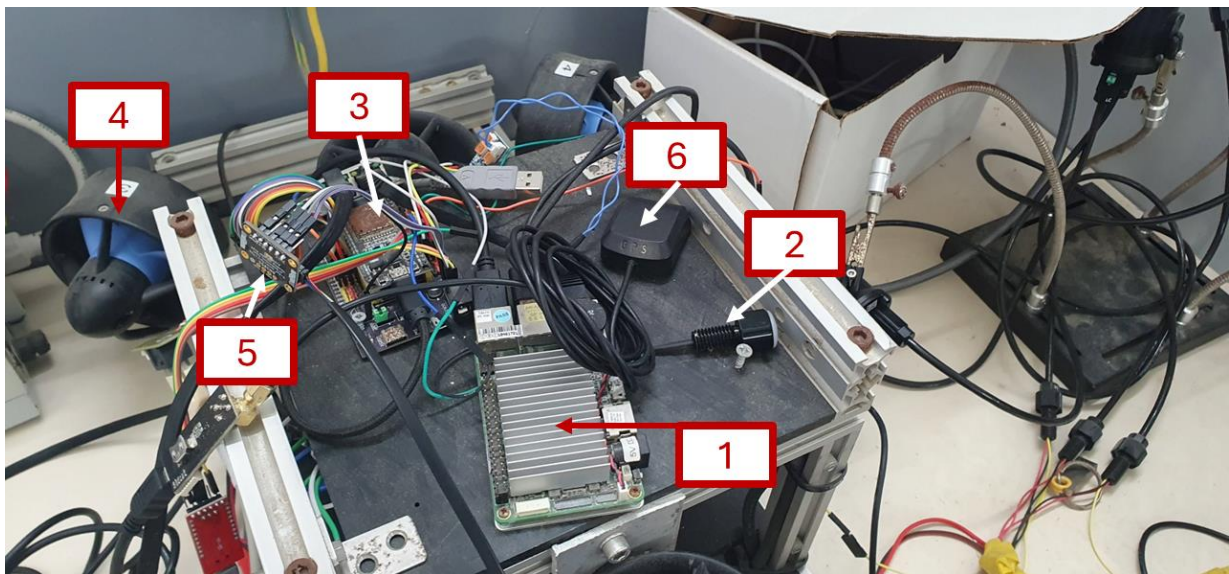


Figure B — The electronics tray. The numbers match the table below.

#	Part	What it is for
1	UP Board	The “brain” — the board with the big metal heatsink.
2	Indicator lights	2 lights, red and green next to each other. See Chapter 10 .
3	ESP32	The “muscles” — the board with the coloured wires.
4	Thruster	One of the propellers.
5	I ² C connector	Goes to the temperature sensor.
6	GPS antenna	The small black square marked GPS. Gives the position on the surface.

What you need before you start

1. The HAUV vehicle, charged.
2. A laptop with **QGroundControl** installed. We call it **QGC**.
3. An Xbox-style game controller, plugged into the laptop.

4. A network cable (for the cable mode), *or* the two Subsonus modems (for the sound mode).

Safety First

Warning! The propellers can cut your fingers. They can start turning at any time when the vehicle is powered on. Keep hands, hair and clothes away.

The golden rules

1. **Never** stand next to the propellers when the vehicle is powered on.
2. **Always** power the vehicle off before you carry it.
3. Do **not** open the vehicle when it is wet. Dry the outside first.
4. Do **not** run the thrusters in air for a long time. They need water to stay cool.
5. If you see water inside, stop. Take the vehicle out. Dry it. Find the leak.
6. Do not charge a battery that is hot or damaged.
7. One person drives. One person watches the vehicle and the cable.

Before every dive

1. Look at the vehicle. Are all screws in place? Is anything broken?
2. Check that all plugs are pushed in fully.
3. Check that the lid is closed and tight.
4. Check the battery is charged.
5. Check the propellers turn freely and nothing is stuck in them.

Note. If anything looks wrong, do not dive. It is always cheaper to fix it on the table than at the bottom of the sea.

Network Setup

The laptop, the vehicle and the sensors all talk over a network. Each one has its own address. The addresses must be correct or nothing will connect.

The addresses

Device	Address	Notes
Your laptop (QGC)	192.168.168.100	You set this by hand.
UP Board (the brain)	192.168.168.101	Fixed. Do not change.
DVL	192.168.168.102	Fixed. Do not change.
Subsonus on the surface	168.254.1.80	Only for the sound mode.

Setting your laptop address

1. Open **Network settings** on your laptop.
2. Find the wired (Ethernet) connection.
3. Choose **Manual / Static** address.
4. Type address **192.168.168.100** and mask **255.255.255.0**.
5. Save.

Tip. To test it, open a terminal and type **ping 192.168.168.101**. If you get replies, the network is good. If you get nothing, check the cable and the address.

Connecting QGroundControl

There are two ways to talk to the HAUV:

Way	When to use it
Cable (Ethernet)	Normal work. Fast. You get video and all the numbers.
Sound (Acoustic / USBL)	When the vehicle is under water with no cable. Slow. No video. Only the important numbers.

Connecting with a cable (Ethernet)

This is the normal way. Do this first when you are learning.

1. Plug the network cable between the laptop and the vehicle.
2. Power the vehicle on.
3. Open **QGroundControl** on the laptop.
4. Click the **Q** logo in the top left corner, then **Application Settings**.
5. Choose **Comm Links** on the left.
6. Click **Add**.
7. Fill in the window exactly as shown in Figure [C](#):
 - **Name:** HAUV UP Board
 - **Type:** UDP
 - **Port:** 14550
 - **Server Address:** 192.168.168.101:14550, then press **Add Server**.
8. Click **Save**.
9. Select HAUV UP Board in the list and click **Connect**.

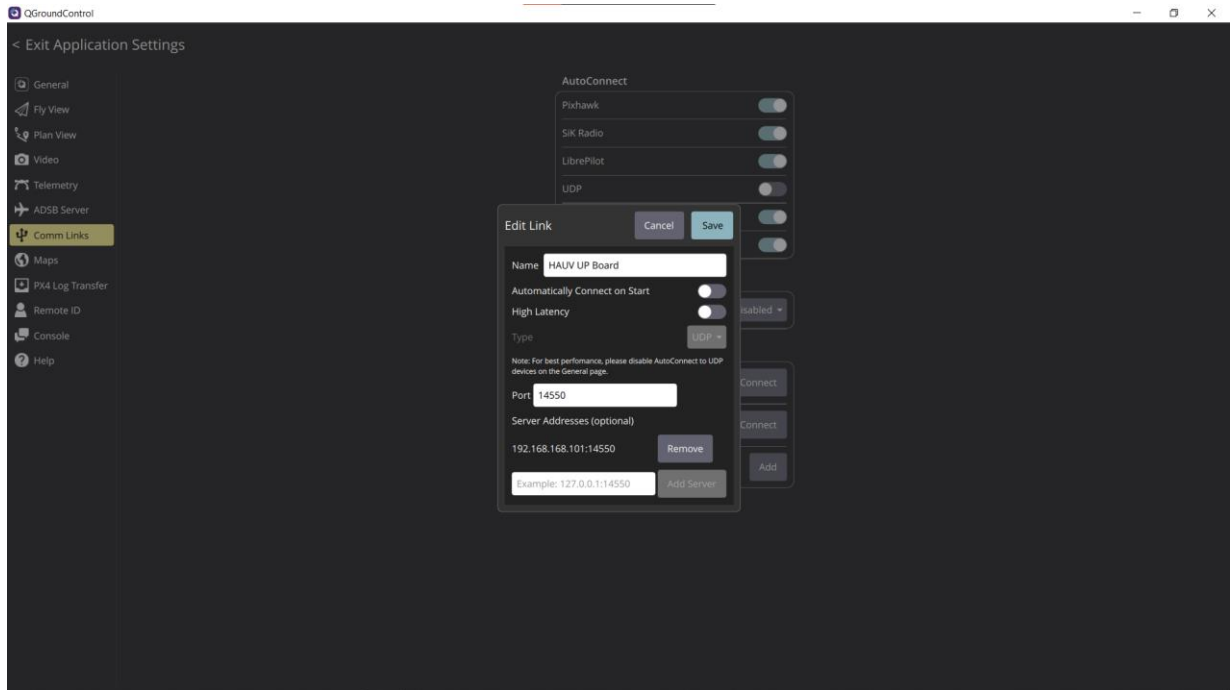


Figure C — The QGC “Edit Link” window. Name, port 14550 and the server address 192.168.168.101:14550 must match exactly.

Note. Turn **AutoConnect to UDP** off on the **General** page. If you leave it on, QGC may connect to the wrong thing and the vehicle will look dead.

When it works you will see the vehicle appear on the map and the numbers at the bottom of the screen will start moving.

Connecting with sound (Acoustic / USBL)

Use this when the vehicle is under water and there is no cable. It is slower. You will **not** get video.

You need two Subsonus modems: one on the surface (in the water, next to you) and one on the vehicle.

1. Put the surface modem in the water. It must be in the water to work.
2. Power both modems on.
3. Open the surface modem page in a web browser. Check the settings match Figure [D](#) (address 168.254.1.80).
4. On the vehicle, start the system in sound mode (see Section [5.3](#)).
5. On the laptop, run the bridge program **once**:

```
python3 tools/acoustic_qgc_bridge.py
```
6. In QGC, connect to 127.0.0.1:14551.

Ethernet Configuration

Current IP Address: 168.254.1.80

Unit Name: [Set To Default](#)

Ethernet Mode: [Set To Default](#)

Fixed IP Address: [Set To Default](#)

Netmask: [Set To Default](#)

Default Gateway: [Set To Default](#)

Broadcast Address: [Set To Default](#)

DNS Server Address: [Set To Default](#)

Link Mode: [Set To Default](#)

[Update Network Settings](#)

Copyright 2017 Advanced Navigation

Figure D — The Subsonus surface unit settings page. The address must be 168.254.1.80.

Warning! Run only **one** copy of the bridge program. If you start it twice, the commands fight each other and the vehicle behaves strangely.

Warning! Both modems must be in the water. Out of the water they cannot hear each other and you will see **ACOUSTIC LINK LOST** on the screen.

DVL settings (for reference only)

The DVL is already set up. You do not normally touch it. Figure E shows the correct settings, in case you ever need to check them.

Pathfinder uDVL Network Configuration [Update](#) Software Version v1.00

Network Settings

Network Configuration **Link Status**

Base IP Address: **Not Active**

Network Mask:

Gateway Address:

Name Server Address:

Command/Control Port:

Output Formats

FORMAT	ENABLE	PROTOCOL	ADDRESS	PORT
PD0	ON	UDP	192.168.168.1	1034
PD4	OFF	UDP	192.168.168.1	1035
PD5	OFF	TCP	192.168.168.1	1036
PD6	ON	UDP	192.168.168.101	1037
PD13	OFF	TCP	192.168.168.1	1038

[Submit](#) [Reset](#)

Figure E — DVL network page. Address 192.168.168.102, command port 1033, outputs PD0 and PD6 switched ON.

Starting the HAUV

The batteries

The HAUV runs on two 12 V lithium battery packs, shown in Figure F. One pack feeds the computers, the other feeds the thrusters. The thick red and black wires join them to the vehicle with yellow plugs.



Figure F — The two 12 V battery packs and how they connect. Red is plus (+), black is minus (-). The small green board is the voltage converter that drops 12 V down for the electronics.

Connecting a battery:

1. Check the vehicle is switched **off** first.
2. Match the colours: red to red, black to black.
3. Push the yellow plug straight in until it clicks. Do not force it sideways.
4. Check nothing is loose and no bare metal is showing.

Warning! Never join red to black. A short circuit makes a big spark, ruins the battery and can start a fire.

Warning! Do not charge a battery that is hot, swollen or damaged. Do not leave batteries charging without somebody nearby.

Note. Charge both packs after every dive. A pack left flat for a long time can be damaged for good.

Power on

1. Check Chapter 2 again. Hands away from the propellers.
2. Switch the vehicle on.

3. Wait about 30 seconds. The computers need time to wake up.

Starting the software

All the software is started with one tool called `hauv.sh`. You run it on the UP Board (the vehicle's brain), through a terminal.

Command	What it does
<code>./hauv.sh start</code>	Starts everything, cable mode. This is the normal one.
<code>./hauv.sh start --acoustic</code>	Starts everything, sound mode (no cable).
<code>./hauv.sh stop</code>	Stops everything.
<code>./hauv.sh restart</code>	Stops, then starts again.
<code>./hauv.sh check</code>	Tests every sensor and says OK, LOW, FAIL or --.
<code>./hauv.sh status</code>	Shows what is running.
<code>./hauv.sh topics</code>	Lists the live data channels.
<code>./hauv.sh echo DEPTH</code>	Shows live values, for example the depth.
<code>./hauv.sh view mavlink</code>	Opens the log of one program. Press Ctrl+A then D to leave.
<code>./hauv.sh help</code>	Shows help and the short names.

Note. The first time you start the HAUV and connect QGC, it does not open on the main screen. It opens on the Vehicle Configuration screen and shows an error that parameters are missing from the firmware (see Figure G). This happens because QGC thinks the vehicle is running ArduSub. It is not a problem. Click OK, then go back to the main screen (the Fly View).

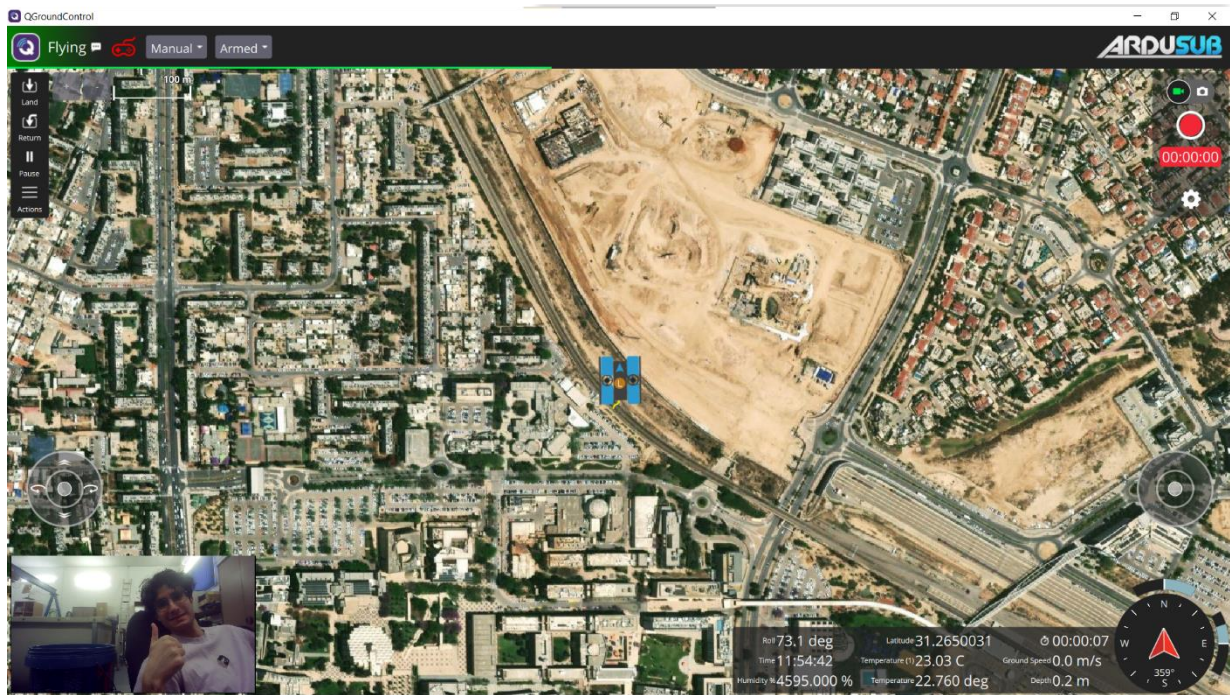


Figure H — The QGC Fly View. The yellow bar at the top is a warning message — here it says ACOUSTIC LINK LOST.

The parts of the screen

Where	What it shows
Middle	The map. The small vehicle symbol is your HAUV.
Top, yellow or red bar	Warning messages. Read these. See Section 6.3 .
Bottom left	The camera picture. Says “WAITING FOR VIDEO” if there is none.
Bottom right	The numbers: depth, roll, temperature, speed, position, time.
Far bottom right	The compass. Shows which way the vehicle points.
Left side	Buttons: Land, Return, Pause, Actions.
Top right	Video record and photo buttons, and the settings gear.

The numbers you must watch

Number	Why it matters
Depth	How deep you are, in metres. Watch this all the time.
Roll	How much the vehicle is leaning. A big number means it is tipping over.
Ground Speed	How fast you are moving.
Latitude / Longitude	Where you are on the map (surface only).
Temperature	The water temperature.

Warning messages

Messages appear in a coloured bar at the top of the screen. Red and yellow ones are important.

Message	What to do
LEAK! Auto-surfacing	Most serious. Water is inside. The vehicle is already swimming up by itself. Get it out of the water. See Section 9.1 .
ACOUSTIC LINK LOST	The sound link stopped. Check both modems are in the water and powered. See Chapter 12 .
FAIL: . . .	A sensor stopped working. The name after FAIL is the sensor.

Note. The leak message repeats every 3 seconds while the leak is there, so you cannot miss it.

Driving the HAUV

The controller

Hold the controller with both hands. The two sticks do the moving. The numbers in Figure I match the table below.

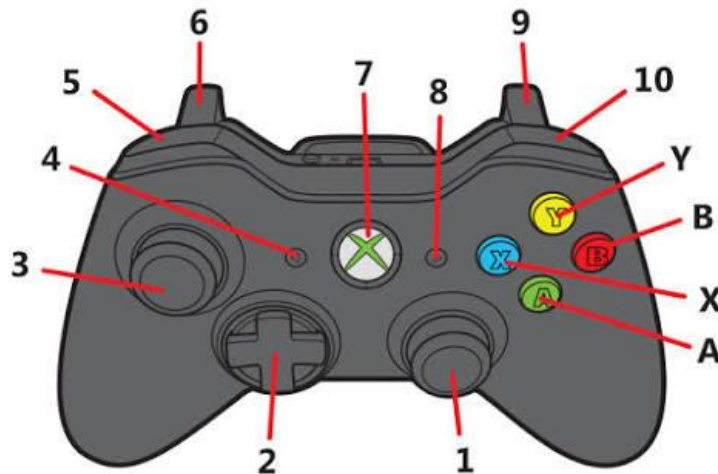


Figure I — The controller. Only the parts listed in the table do anything on the HAUV — the rest are not used.

#	Control	What it does on the HAUV
3	Left stick	Up / down: move forward and backward. Left / right: slide sideways (crab). The nose keeps pointing the same way.
1	Right stick	Left / right: turn left and right. Up / down: go up and down in the water.
A	A button	Switch one set of lights on and off.
B	B button	Switch the other set of lights on and off.
5	LB shoulder	Tilt the camera up.
10	RB shoulder	Tilt the camera down.

The parts **not** used for driving are: 2 (D-pad), 4 (Back), 6 and 9 (the two triggers), 7 (Xbox button), 8 (Start), and the X and Y buttons.

Tip. Push the sticks gently. Small movements are enough. The vehicle is heavy and keeps moving after you let go.

Your first drive

1. Put the vehicle in the water. Hold it until you are ready.
2. Check the screen. The depth should be near 0.
3. Push the left stick forward a little. The vehicle should move forward.

4. Pull it back. The vehicle should stop and go back.
5. Turn with the right stick. Watch the compass move.
6. Push the right stick down a little to go deeper. Watch the depth number grow.

Warning! Always know where the vehicle is. If you cannot see it and the screen is frozen, stop and bring it back.

Flight Modes

The vehicle has three modes. You change them in QGC, at the top of the screen.

Manual

You control everything with the controller. The vehicle does only what you tell it. Use this most of the time.

Stabilize

The vehicle tries to stay level by itself. You still drive it. Use this if the vehicle is rolling or tipping too much.

Guided — “Go to location”

The vehicle drives itself to a point you choose on the map. It keeps its depth on the way, and when it arrives it stays there instead of drifting away.

How to use it:

1. Make sure the vehicle has a GPS position (it must be on the surface).
2. On the map, click the place you want to go.
3. Choose **Go to location**.
4. The vehicle turns towards the point, then swims to it.
5. When it arrives, it stays there and holds its position and depth.

How to stop it (any of these work):

1. Move any stick on the controller.
2. Change the mode to **Manual** in QGC.
3. If the link is lost for 5 seconds, the vehicle stops by itself.

Note. While going to the point, the vehicle keeps the depth it had when you pressed **Go to location**. If you were on the surface, it stays on the surface.

Safety Features

These work by themselves. You do not switch them on.

Leak — the vehicle swims up by itself

If water gets inside, the leak sensor sees it. Then:

1. The vehicle **stops** all sideways movement.
2. It drives its up/down thrusters to **go up** to the surface.
3. It stays on the surface.
4. Your screen shows **LEAK! Auto-surfacing** in red, again and again.

This ignores everything else. Even if you are driving, or the vehicle is going to a point by itself, the leak wins and the vehicle goes up.

What you must do:

1. Do not fight it. Let it come up.
2. Take the vehicle out of the water.
3. Dry the outside. Then open it and find the water.
4. Do not dive again until you find why water came in.

Note. This works even if the cable or the sound link is broken. The decision is made inside the vehicle, not on your laptop.

Note. When the sensor is dry again for 5 seconds, the vehicle gives control back to you automatically. If the sensor stays wet, it stays on the surface — that is on purpose.

Lost control link

If the vehicle stops hearing your controller for about 1.5 seconds, it sets all thrusters to stop. It will not keep driving away with the last command.

Indicator Lights

There is a **green** light indicator. It tells you if the two computers are talking to each other. You can see where it sits in Figure [B](#) — it is number **2**.

Green light	Meaning
On, steady	Good. The two computers are talking. Sensors and motors work.
Off	Bad. They are not talking. The motors will not answer. See Chapter 12 .

Note. Just after you switch the vehicle on, the green light is on for a short time before the connection is really made. Wait about 30 seconds before you trust it.

A **red** light indicator sits right next to the green one and shows leaks, faults, and when the propellers are live — keep clear of the propellers whenever it is on (see Chapter 2).

Red light	Meaning
Fast Blink	Leak Detected
Solid	Sensor Fault
Slow Blink	Props Live – Keep Clear
Off	All Clear

After the Dive

1. Bring the vehicle out of the water. Two people if it is heavy.
2. Switch the vehicle off.
3. Wash the outside with **fresh** water. Salt water eats metal.
4. Pay attention to the propellers. Remove weed, sand and plastic.
5. Dry the vehicle with a cloth.
6. Only when it is dry outside, open it and check for water inside.
7. Charge the battery.

Warning! Never put the vehicle away wet or with a flat battery.

Troubleshooting

Common problems

Problem	What to try
QGC does not connect	1. Check the network cable. 2. Check your laptop address is 192.168.168.100. 3. ping 192.168.168.101. 4. Check the link settings match Figure C . 5. Turn AutoConnect UDP off in QGC.
Vehicle shows on screen but nothing moves	The two computers are not talking. Check the green light (Chapter 10). Restart with <code>./hauv.sh restart</code> .
No video picture	Video only works over the cable. Check the cable mode is running.
DVL says FAIL	Normal out of the water. If it still fails in the water, check the DVL settings in Figure E .
ACOUSTIC LINK LOST	1. Both modems must be <i>in the water</i> . 2. Check both are powered. 3. Check only <i>one</i> bridge program is running on the laptop. 4. Check the surface modem address (Figure D).
Some sensors say FAIL	Run <code>./hauv.sh check</code> to see which. Restart with <code>./hauv.sh restart</code> . If it still fails, the sensor or its plug is bad.

Problem	What to try
Vehicle drifts away in Go to location	The GPS is only accurate to a few metres. This is normal. Take manual control if you need to be exact.
Vehicle keeps swimming up and will not stop	This is the leak failsafe. There is water inside. See Section 9.1 . Get it out of the water.

Useful checks

Command	Use it to
<code>./hauv.sh check</code>	See which sensors are alive.
<code>./hauv.sh status</code>	See which programs are running.
<code>./hauv.sh echo DEPTH</code>	Watch the depth value live.
<code>./hauv.sh echo LEAK</code>	Check the leak sensor (0 = dry, 1 = water).
<code>./hauv.sh view guidance</code>	Read the messages from the driving program.
<code>./hauv.sh restart</code>	The general “switch it off and on again”.

Tip. If you are lost, `./hauv.sh restart` fixes most software problems. It does not fix broken cables or wet electronics.